

Lecture 3

Autoregressive Model

Topics in Generative AI

Dr. Jianyuan Guo

Fall 2025, Dept. CS, CityU



香港城市大學
City University of Hong Kong
Innovating into the Future

Overview



- Representing joint probability distributions
 - Naive Bayes for single label prediction
 - Neural models
- Autoregressive Models
 - FVSBN (Fully Visible Sigmoid Belief Network)
 - NADE (Neural Autoregressive Density Estimation)
 - LLM

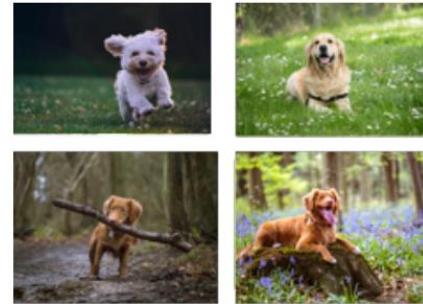
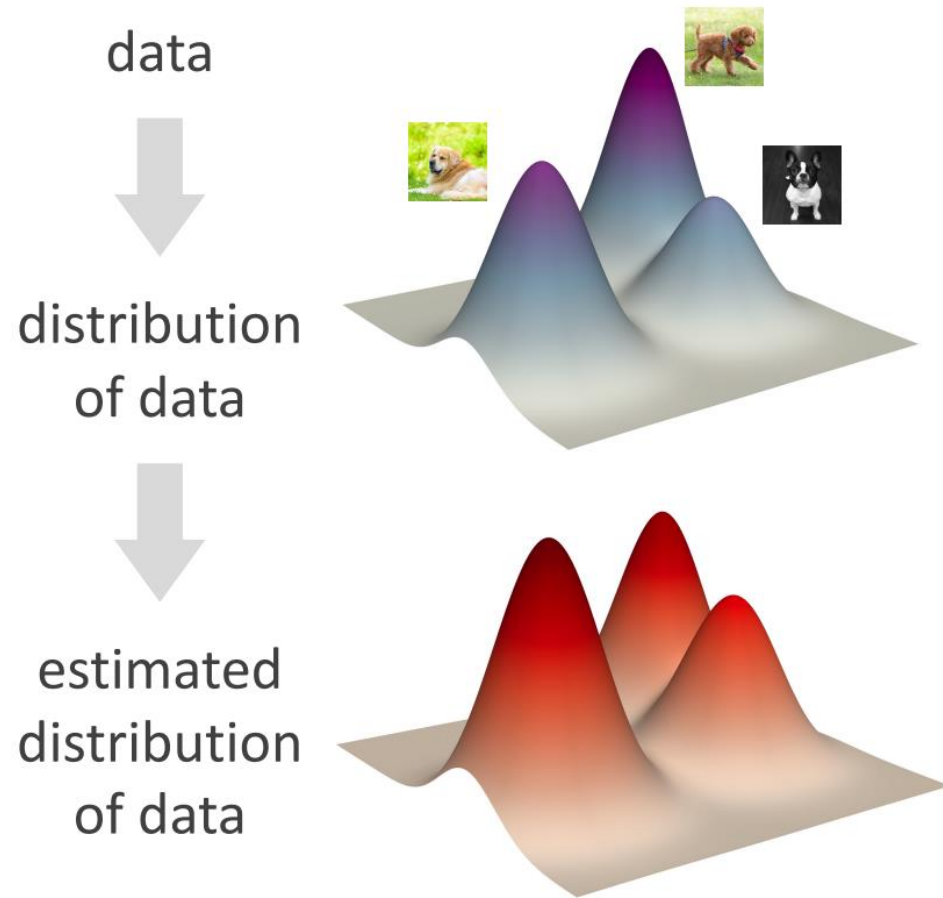
Acknowledgement

This presentation is based on materials from:

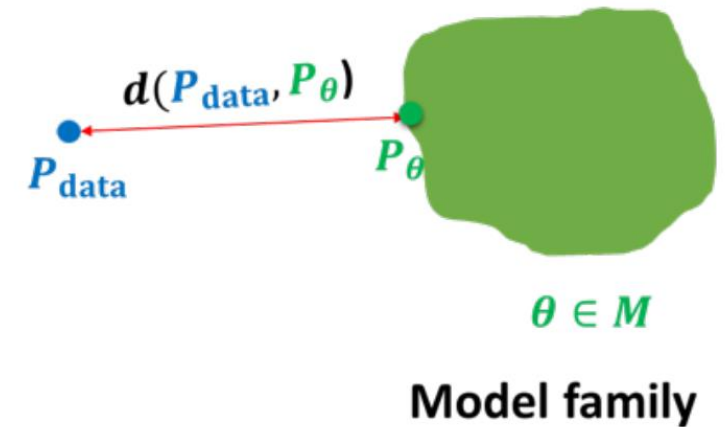
- [6.S978: Deep Generative Models] by [Kaiming He/MIT], [2024]
- [CS236 : Deep Generative Models] by [Stefano Ermon/Stanford], [2023]

The slides have been adapted and extended with additional content, examples, and illustrations for the purpose of this lecture.

Recap: Probabilistic Modeling



$$x_i \sim P_{\text{data}} \\ i = 1, 2, \dots, n$$

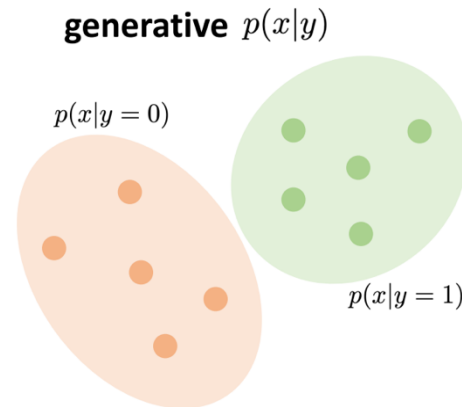
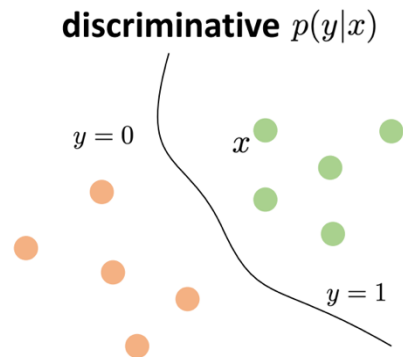


We want to learn a probability distribution $p(x)$ over images x

- **Generation:** If we sample $x_{\text{new}} \sim p(x)$, x_{new} should look like a dog (sampling)
- **Density estimation:** $p(x)$ should be high if x looks like a dog, and low otherwise (anomaly detection)

- Optimize a loss function $\mathcal{L}(\text{distribution of data}, \text{estimated distribution of data})$

Recap: Discriminative vs. generative



- Generative models can be discriminative: Bayes' rule

$$\text{discriminative } p(y|x) = \text{generative } p(x|y) \frac{p(y)}{p(x)}$$

← assuming known prior

← constant for given x

- Can discriminative models be generative?

$$\text{generative } p(x|y) = \text{discriminative } p(y|x) \frac{p(x)}{p(y)}$$

← still need to model prior distribution of x

← constant for given y

- The challenge is about representing and predicting distributions

Recap: Logistic regression

- For the discriminative model, assume that

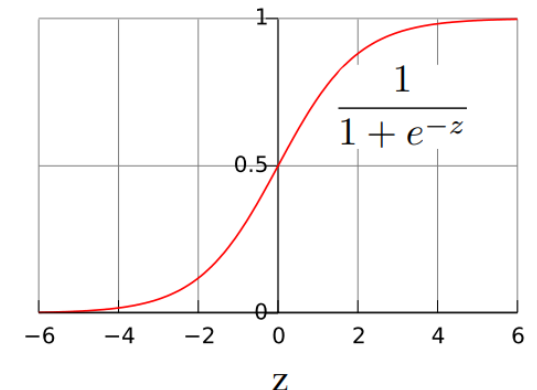
$$p(Y = 1 \mid \mathbf{x}; \boldsymbol{\alpha}) = f(\mathbf{x}, \boldsymbol{\alpha})$$

- Not represented as a table anymore. It is a parameterized function of $\mathbf{x}$ (regression)
 - Has to be between 0 and 1
 - Depend in some simple but reasonable way on $X_1, \dots, X_n$
 - Completely specified by a vector $\boldsymbol{\alpha}$ of $n+1$ parameters (**compact representation**)

Linear dependence: let $z(\boldsymbol{\alpha}, \mathbf{x}) = \alpha_0 + \sum_{i=1}^n \alpha_i x_i$.

Then, $p(Y = 1 \mid \mathbf{x}; \boldsymbol{\alpha}) = \sigma(z(\boldsymbol{\alpha}, \mathbf{x}))$

where $\sigma(z) = 1/(1 + e^{-z})$ is called the **logistic function**:

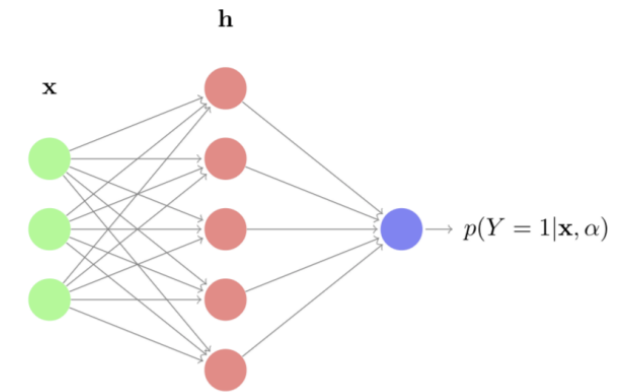


Recap: Neural Models for classification

For classification, we care about $p(Y | \mathbf{x})$, and assume that

$$p(Y = 1 | \mathbf{x}; \alpha) = f(\mathbf{x}, \alpha)$$

Logistic regression: let $z(\alpha, \mathbf{x}) = \alpha_0 + \sum_{i=1}^n \alpha_i x_i$.
 $p_{\text{logit}}(Y = 1 | \mathbf{x}; \alpha) = \sigma(z(\alpha, \mathbf{x}))$, where $\sigma(z) = 1/(1 + e^{-z})$



Non-linear dependence: let $\mathbf{h}(A, \mathbf{b}, \mathbf{x})$ be a non-linear transformation of the input features. $p_{\text{Neural}}(Y = 1 | \mathbf{x}; \alpha, A, \mathbf{b}) = \sigma(\alpha_0 + \sum_{i=1}^h \alpha_i \mathbf{h}_i)$

- More flexible
- More parameters: $A, \mathbf{b}, \alpha$
- Repeat multiple times to get a multilayer perceptron (neural network)

Summary: Represent Joint Distribution

- Using Chain Rule (Fully General)

$$p(x_1, x_2, x_3, x_4) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2)p(x_4 \mid x_1, x_2, x_3)$$

- Bayes Net (Assumes conditional independencies)

$$p(x_1, x_2, x_3, x_4) \approx p(x_1)p(x_2 \mid x_1)p(x_3 \mid \cancel{x_1}, x_2)p(x_4 \mid \cancel{x_1}, \cancel{x_2}, x_3)$$

- Neural Models (Assume specific functional form for the conditionals. A sufficiently deep neural net can approximate any function.)

$$p(x_1, x_2, x_3, x_4) \approx p(x_1)p(x_2 \mid x_1)p_{\text{Neural}}(x_3 \mid x_1, x_2)p_{\text{Neural}}(x_4 \mid x_1, x_2, x_3)$$

Generative Model Example: MNIST

- **Given:** a dataset D of handwritten digits (binarized MNIST)



- Each image has $n = 28 \times 28 = 784$ pixels. Each pixel can either be black (0) or white (1).
- **Goal:** Learn a probability distribution $p(x) = p(x_1, \dots, x_{784})$ over $x \in \{0,1\}^{784}$ such that when $x \sim p(x)$, x looks like a digit
- Two step process:
 - Parameterize a model family $\{p_\theta(x), \theta \in \Theta\}$
 - Search for model parameters θ based on training data $\mathcal{D}$

Autoregressive Models

- We can pick an ordering of all the random variables, i.e., raster scan ordering of pixels from top-left (X_1) to bottom-right ($X_{n=784}$)

- Without loss of generality, we can use chain rule for factorization

$$p(x_1, \dots, x_{784}) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2) \cdots p(x_n \mid x_1, \dots, x_{n-1})$$

- Some conditionals are too complex to be stored in tabular form. We **assume**

$$p(x_1, \dots, x_{784}) = p_{\text{CPT}}(x_1; \alpha^1) p_{\text{logit}}(x_2 \mid x_1; \alpha^2) p_{\text{logit}}(x_3 \mid x_1, x_2; \alpha^3) \cdots p_{\text{logit}}(x_n \mid x_1, \dots, x_{n-1}; \alpha^n)$$

$$p_{\text{CPT}}(X_1 = 1; \alpha^1) = \alpha^1, \quad p(X_1 = 0) = 1 - \alpha^1$$

$$p_{\text{logit}}(X_2 = 1 \mid x_1; \alpha^2) = \sigma(\alpha_0^2 + \alpha_1^2 x_1)$$

$$p_{\text{logit}}(X_3 = 1 \mid x_1, x_2; \alpha^3) = \sigma(\alpha_0^3 + \alpha_1^3 x_1 + \alpha_2^3 x_2)$$

Autoregressive Models

- Chain rule

$$p(x_1, \dots, x_{784}) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2) \cdots p(x_n \mid x_1, \dots, x_{n-1})$$

- Logistic regression to estimate $p(X_n)$

$$p(x_1, \dots, x_{784}) = p_{\text{CPT}}(x_1; \alpha^1) p_{\text{logit}}(x_2 \mid x_1; \alpha^2) p_{\text{logit}}(x_3 \mid x_1, x_2; \alpha^3) \cdots p_{\text{logit}}(x_n \mid x_1, \dots, x_{n-1}; \alpha^n)$$

$$p_{\text{CPT}}(X_1 = 1; \alpha^1) = \alpha^1, \quad p(X_1 = 0) = 1 - \alpha^1$$

$$p_{\text{logit}}(X_2 = 1 \mid x_1; \alpha^2) = \sigma(\alpha_0^2 + \alpha_1^2 x_1)$$

$$p_{\text{logit}}(X_3 = 1 \mid x_1, x_2; \alpha^3) = \sigma(\alpha_0^3 + \alpha_1^3 x_1 + \alpha_2^3 x_2)$$

- Note: This is a modeling **assumption**. We are using parameterized functions (e.g., logistic regression) to predict next pixel given all the previous ones (**autoregressive model**).

Fully Visible Sigmoid Belief Network

- The conditional variables $X_i \mid X_1, X_2, \dots, X_{i-1}$ are Bernoulli with parameters

$$\hat{x}_i = p(X_i = 1 \mid x_1, \dots, x_{i-1}; \alpha^i) = p(X_i = 1 \mid x_{<i}; \alpha^i) = \sigma(\alpha_0^i + \sum_{j=1}^{i-1} \alpha_j^i x_j)$$

- How to evaluate $p(X_1, \dots, X_{784})$? Multiply all the conditionals (factors)

$$\begin{aligned} p(X_1 = 0, X_2 = 1, X_3 = 1, X_4 = 0) &= (1 - \hat{x}_1) \times \hat{x}_2 \times \hat{x}_3 \times (1 - \hat{x}_4) \\ &= (1 - \hat{x}_1) \times \hat{x}_2(X_1 = 0) \times \hat{x}_3(X_1 = 0, X_2 = 1) \times (1 - \hat{x}_4(X_1 = 0, X_2 = 1, X_3 = 1)) \end{aligned}$$

- How to sample from $p(X_1, \dots, X_{784})$?

Sample $\bar{x}_1 \sim p(x_1)$ (`np.random.choice([1,0], p=[ $\hat{x}_1$ ,  $1 - \hat{x}_1$ ])`)

Sample $\bar{x}_2 \sim p(x_2 \mid x_1 = \bar{x}_1)$

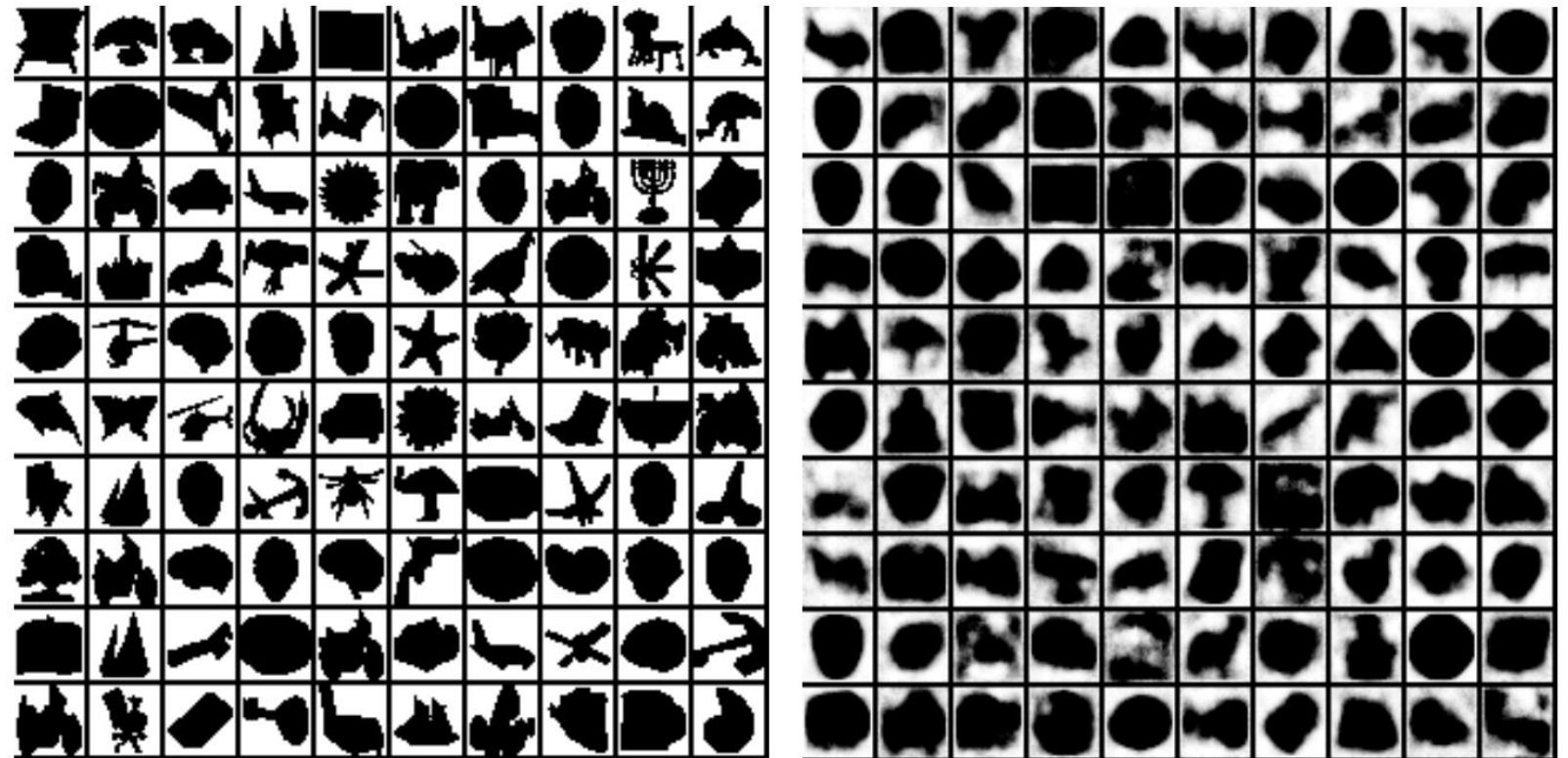
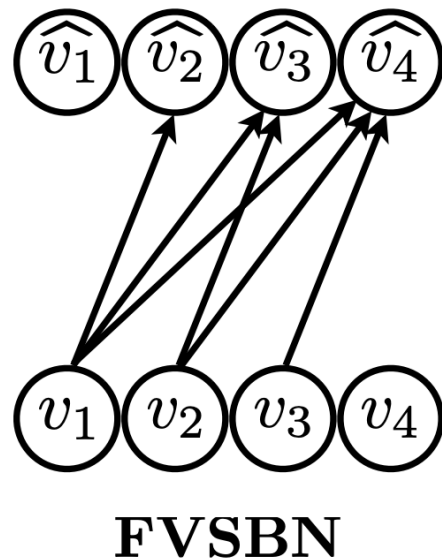
Sample $\bar{x}_3 \sim p(x_3 \mid x_1 = \bar{x}_1, x_2 = \bar{x}_2) \dots$

- How many parameters (in the α^i vectors)? $1 + 2 + 3 + \dots + n \approx n^2/2$

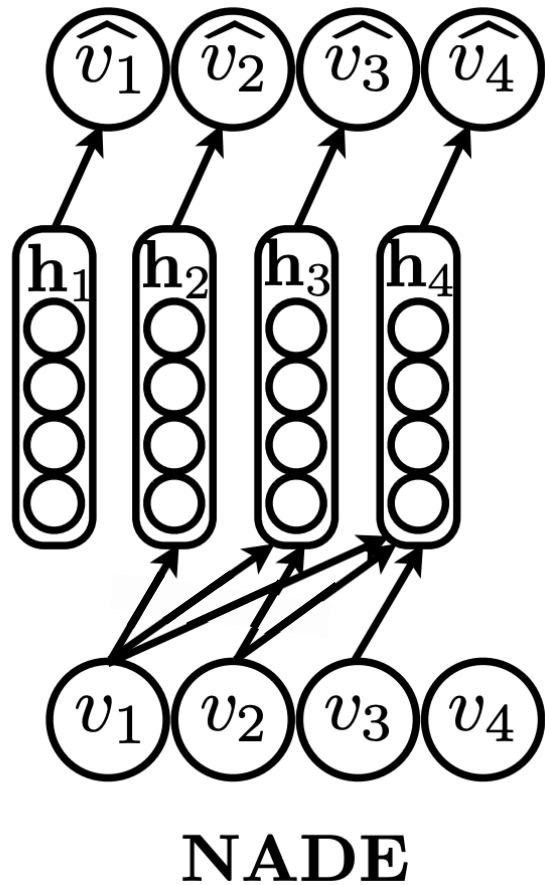
FVSBN Results

Performance on the Caltech 101 Silhouettes dataset.

- Left: Training data.
- Right: Samples from the model.



Neural Autoregressive Density Estimation



- To improve model: use **one-layer neural network** instead of logistic regression

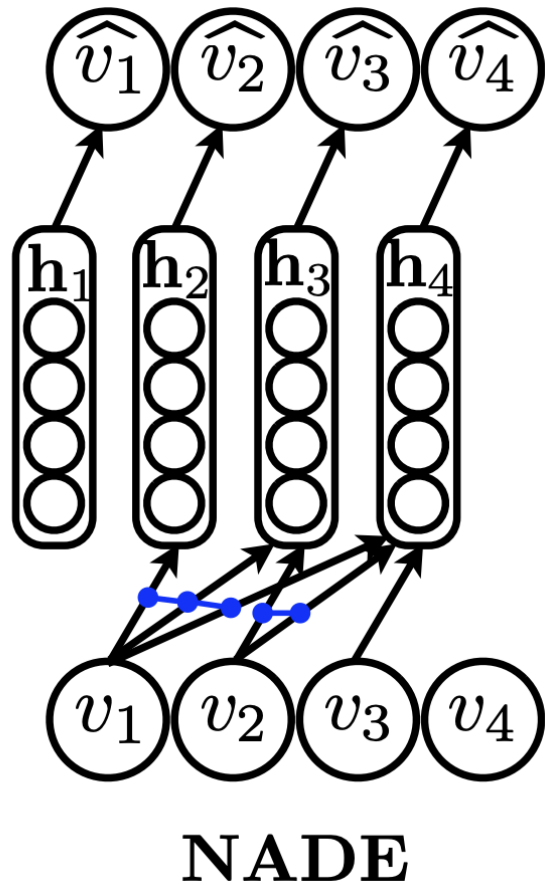
$$\mathbf{h}_i = \sigma(A_i \mathbf{x}_{<i} + \mathbf{c}_i)$$

$$\hat{x}_i = p(x_i | x_1, \dots, x_{i-1}; \underbrace{A_i, \mathbf{c}_i, \boldsymbol{\alpha}_i, b_i}_{\text{parameters}}) = \sigma(\boldsymbol{\alpha}_i \mathbf{h}_i + b_i)$$

- For example:

$$\mathbf{h}_2 = \sigma \left(\underbrace{\begin{pmatrix} \vdots \\ \vdots \end{pmatrix}}_{A_2} x_1 + \underbrace{\begin{pmatrix} \vdots \\ \vdots \end{pmatrix}}_{c_2} \right) \quad \mathbf{h}_3 = \sigma \left(\underbrace{\begin{pmatrix} \vdots \\ \vdots \\ \vdots \end{pmatrix}}_{A_3} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \underbrace{\begin{pmatrix} \vdots \\ \vdots \end{pmatrix}}_{c_3} \right)$$

Neural Autoregressive Density Estimation



- Tie weights to reduce the number of parameters and speed up computation (see blue dots in the figure):

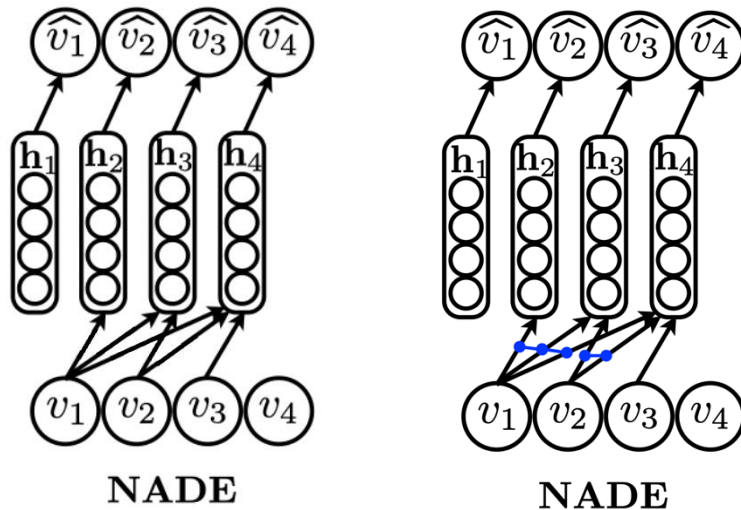
$$\mathbf{h}_i = \sigma(W_{\cdot, < i} \mathbf{x}_{< i} + \mathbf{c})$$

$$\hat{x}_i = p(x_i | x_1, \dots, x_{i-1}) = \sigma(\alpha_i \mathbf{h}_i + b_i)$$

- For example:

$$\mathbf{h}_2 = \sigma \left(\underbrace{\begin{pmatrix} \vdots \\ \mathbf{w}_1 \\ \vdots \end{pmatrix}}_{W_{\cdot, < 2}} x_1 + \mathbf{c} \right) \quad \mathbf{h}_3 = \sigma \left(\underbrace{\begin{pmatrix} \vdots \\ \mathbf{w}_1 & \mathbf{w}_2 \\ \vdots \end{pmatrix}}_{W_{\cdot, < 3}} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \right) \quad \mathbf{h}_4 = \sigma \left(\underbrace{\begin{pmatrix} \vdots & \vdots & \vdots \\ \mathbf{w}_1 & \mathbf{w}_2 & \mathbf{w}_3 \\ \vdots & \vdots & \vdots \end{pmatrix}}_{W_{\cdot, < 4}} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} \right)$$

Neural Autoregressive Density Estimation



$$\mathbf{h}_2 = \sigma \left(\underbrace{\begin{pmatrix} \vdots \\ \vdots \end{pmatrix}}_{A_2} x_1 + \underbrace{\begin{pmatrix} \vdots \\ \vdots \end{pmatrix}}_{c_2} \right) \quad \mathbf{h}_3 = \sigma \left(\underbrace{\begin{pmatrix} \vdots \\ \vdots \end{pmatrix}}_{A_3} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \underbrace{\begin{pmatrix} \vdots \\ \vdots \end{pmatrix}}_{c_3} \right)$$

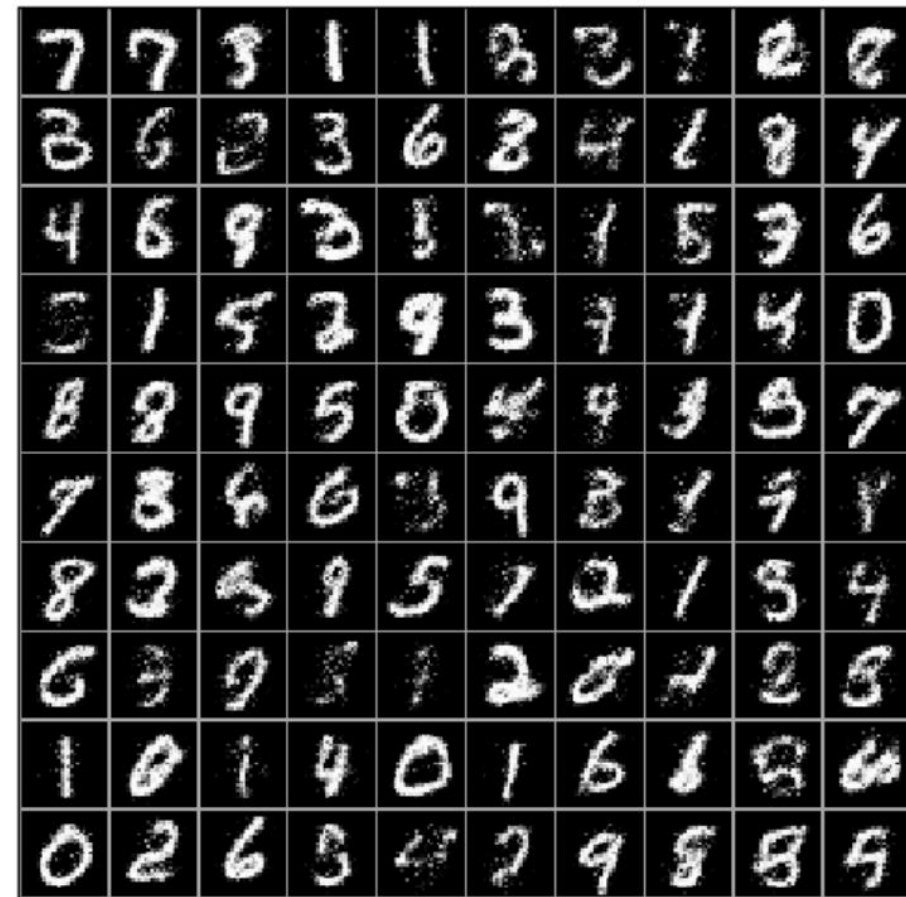
$$\mathbf{h}_2 = \sigma \left(\underbrace{\begin{pmatrix} \vdots \\ \vdots \\ \mathbf{w}_1 \\ \vdots \end{pmatrix}}_{W_{\cdot, < 2}} x_1 + \mathbf{c} \right) \quad \mathbf{h}_3 = \sigma \left(\underbrace{\begin{pmatrix} \vdots \\ \vdots \\ \mathbf{w}_1 & \mathbf{w}_2 \\ \vdots \\ \vdots \end{pmatrix}}_{W_{\cdot, < 3}} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \right) \quad \mathbf{h}_4 = \sigma \left(\underbrace{\begin{pmatrix} \vdots \\ \vdots \\ \mathbf{w}_1 & \mathbf{w}_2 & \mathbf{w}_3 \\ \vdots \\ \vdots \end{pmatrix}}_{W_{\cdot, < 4}} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} \right)$$

If $\mathbf{h}_i \in \mathbb{R}^d$, how many total parameters?

- Linear in n
- weights $\mathbf{W} \in \mathbb{R}^{d \times n}$, biases $\mathbf{c} \in \mathbb{R}^{d \times n}$, and n logistic regression coefficient vectors $\alpha_i, b_i \in \mathbb{R}^{d+1}$. Probability is evaluated in $\mathcal{O}(nd)$.

NADE Results

- Left: Samples from NADE trained on a binary version of MNIST.
- Right: Probabilities from which each pixel was sampled.



NADE Results

- Left: Samples from NADE trained on a binary version of MNIST.
- Right: Probabilities from which each pixel was sampled.

$$\hat{x}_i = p(x_i | x_1, \dots, x_{i-1})$$



General discrete distributions

- How to model non-binary discrete random variables $X_i \in \{1, 2, \dots, K\}$? E.g., pixel intensities varying from 0 to 255
- One solution: Let $\hat{X}_i$ parameterize a categorical distribution

$$\mathbf{h}_i = \sigma(W_{\cdot, < i} \mathbf{x}_{< i} + \mathbf{c})$$

$$p(x_i | x_1, \dots, x_{i-1}) = \text{Cat}(p_i^1, \dots, p_i^K)$$

$$\hat{\mathbf{x}}_i = (p_i^1, \dots, p_i^K) = \text{softmax}(\mathbf{A}_i \mathbf{h}_i + \mathbf{b}_i)$$

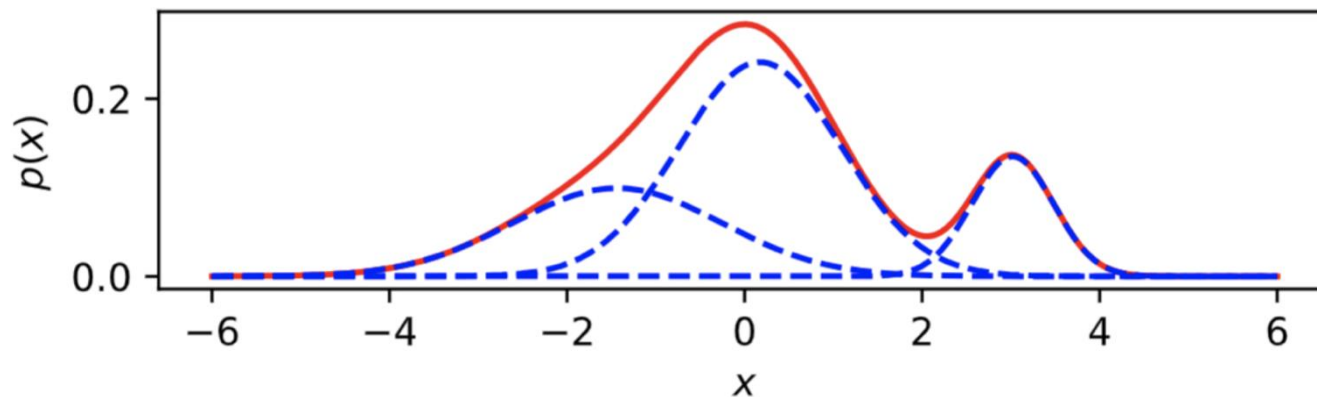
- **Softmax operation:** transforms a vector of K numbers into a vector of K probabilities (non-negative, sum to 1).

$$\text{softmax}(\mathbf{a}) = \text{softmax}(a^1, \dots, a^K) = \left(\frac{\exp(a^1)}{\sum_i \exp(a^i)}, \dots, \frac{\exp(a^K)}{\sum_i \exp(a^i)} \right)$$

- In numpy: `np.exp(a)/np.sum(np.exp(a))`
- In torch: `torch.softmax()`

RNADE: Real-valued Neural Autoregressive Density Estimation

- How to model continuous random variables $X_i \in \mathbb{R}$? E.g., speech signals
- Solution: let $\hat{X}_i$ parameterize a continuous distribution
 - E.g., uniform mixture of K Gaussians



$$p(x_i | x_1, \dots, x_{i-1}) = \sum_{j=1}^K \frac{1}{K} \mathcal{N}(x_i; \mu_i^j, \sigma_i^j)$$

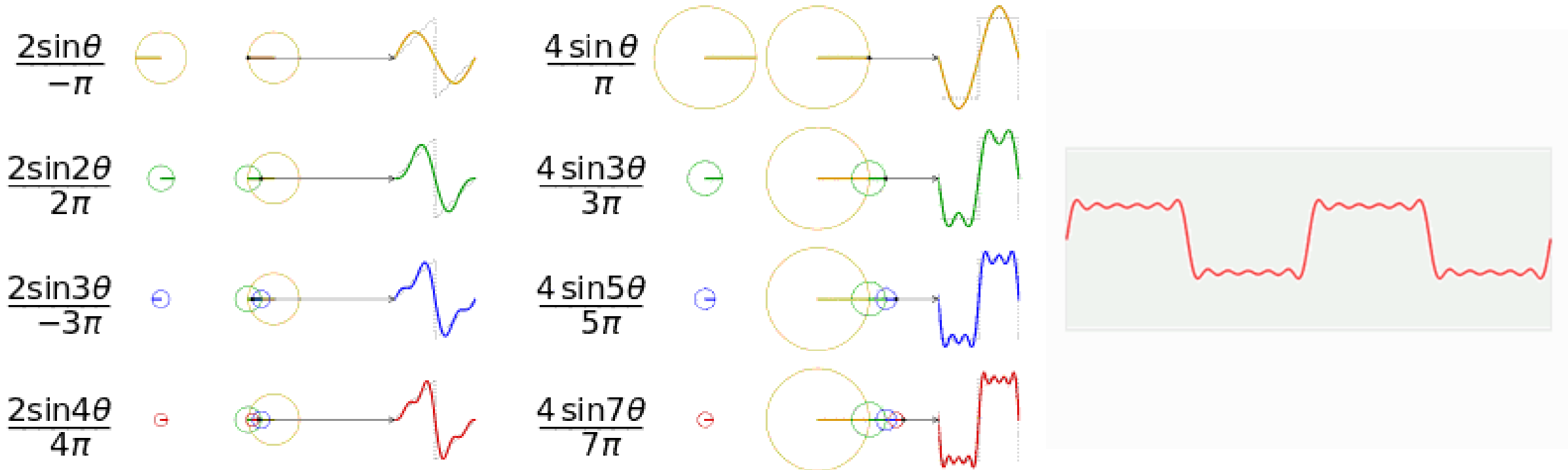
$$\mathbf{h}_i = \sigma(W_{\cdot, < i} \mathbf{x}_{< i} + \mathbf{c})$$

$$\hat{\mathbf{x}}_i = (\mu_i^1, \dots, \mu_i^K, \sigma_i^1, \dots, \sigma_i^K) = f(\mathbf{h}_i)$$

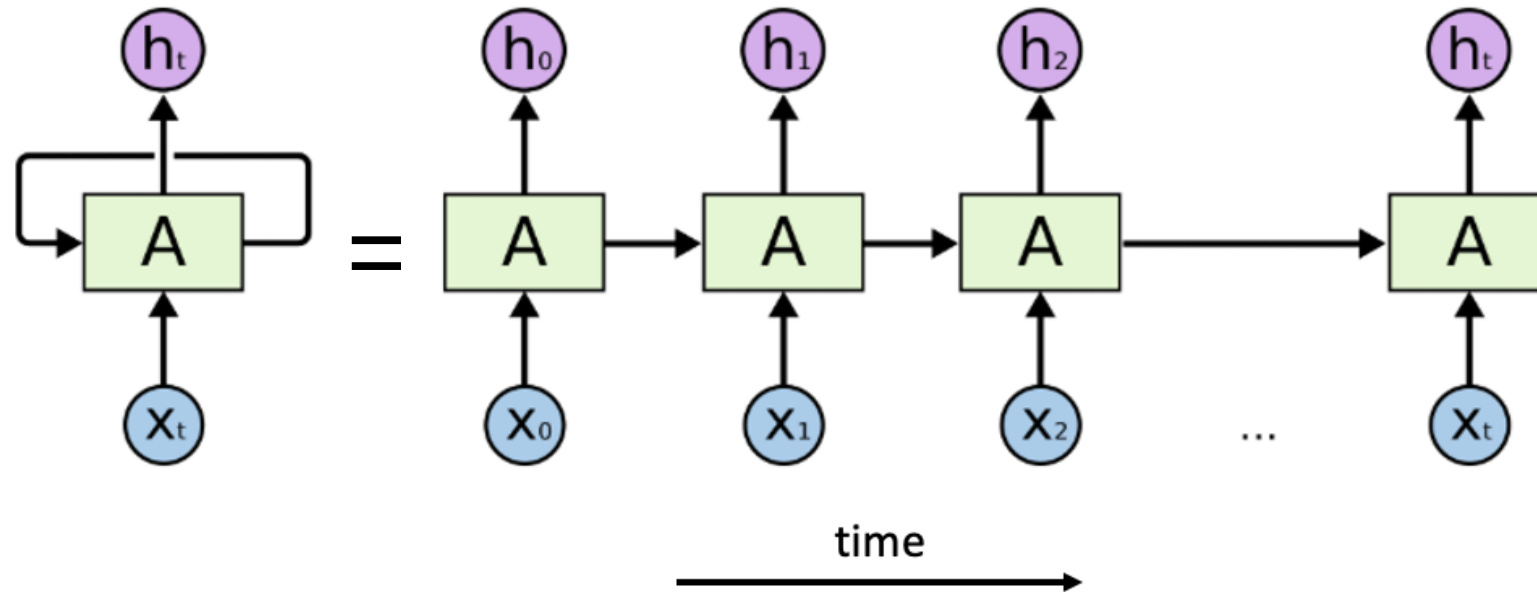
- $\hat{X}_i$ defines the mean and standard deviation of each of the K Gaussians (μ_i^j, σ_i^j) .
- Can use exponential $\exp(\cdot)$ to ensure non-negativity

Fourier Transform

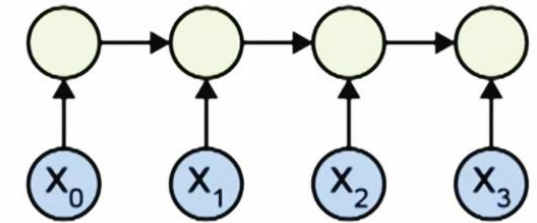
- Any periodic or non-periodic signal (that meets mild mathematical conditions) can be decomposed into a sum (or integral) of sine and cosine waves (pure frequency components) with different amplitudes, frequencies, and phases.



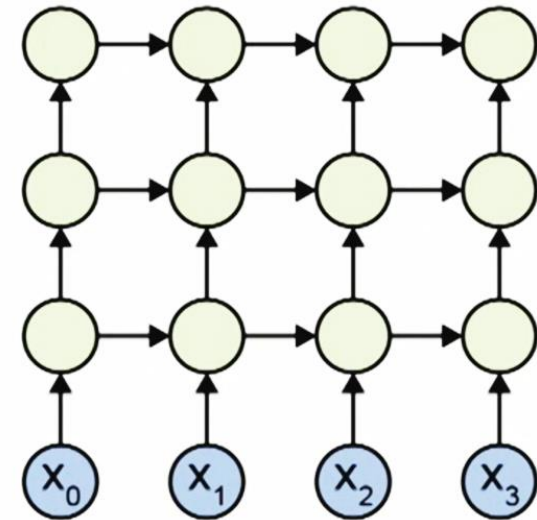
RNN: Recurrent Neural Nets



1-layer RNN



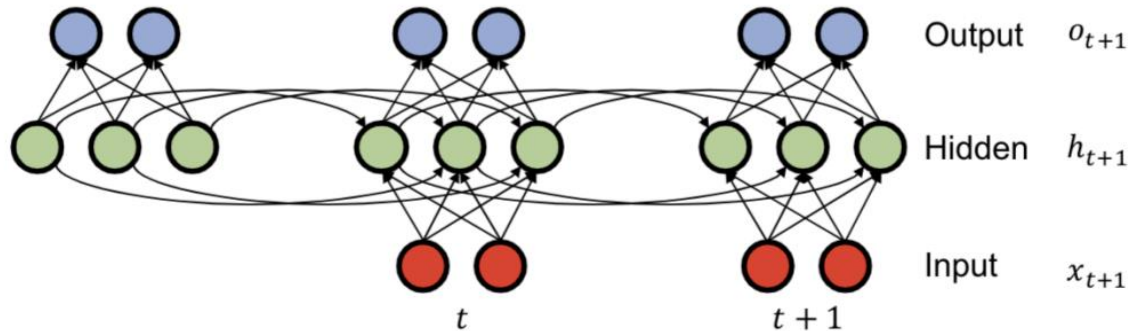
3-layer RNN



time

RNN: Recurrent Neural Nets

- Challenge: model $p(X_t | X_{1:t-1}; \alpha^t)$. “History” $X_{1:t-1}$ keeps getting longer.
- Idea: keep a summary and recursively update it.



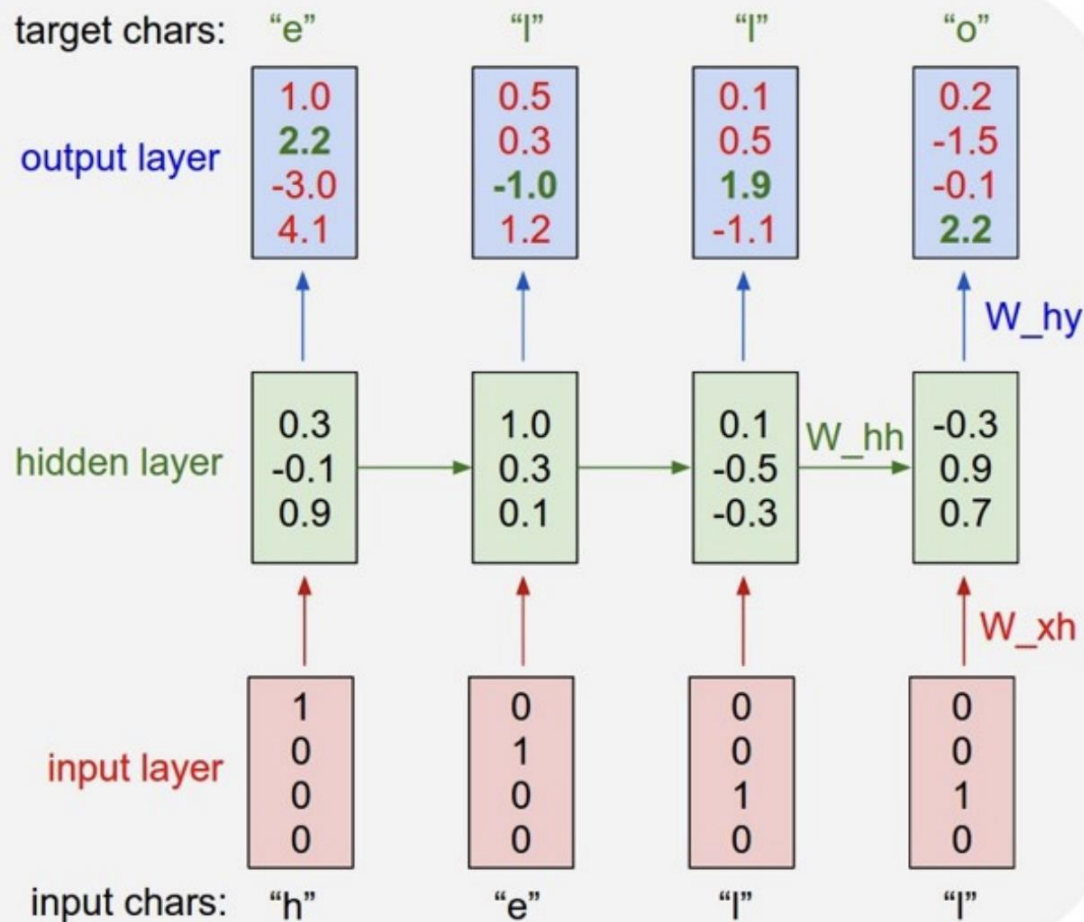
Summary update rule: $h_{t+1} = \tanh(W_{hh}h_t + W_{xh}x_{t+1})$

Prediction: $o_{t+1} = W_{hy}h_{t+1}$

Summary initialization: $h_0 = b_0$

- Hidden layer h_t is a summary of the inputs seen till time t
- Output layer o_t specifies parameters for conditional $p(X_t | X_{1:t-1})$
- Parameterized by b_0 (initialization), and matrices W_{hh} , W_{xh} , W_{hy} .
- Constant number of parameters w.r.t n

Example: Character RNN



- Suppose $X_i \in \{h, e, l, o\}$. Use one-hot encoding:
 - h encoded as $[1, 0, 0, 0]$
 - e encoded as $[0, 1, 0, 0]$
 - etc.

- Autoregressive:

$$P(X = \text{hello}) = P(X_1 = h) \cdot P(X_2 = e | X_1 = h) \cdot$$

$$P(X_3 = l | X_1 = h, X_2 = e) \cdot$$

$$P(X_4 = l | X_1 = h, X_2 = e, X_3 = l) \cdot$$

$$P(X_5 = o | X_1 = h, X_2 = e, X_3 = l, X_4 = l)$$

- For example:

$$p(x_2 = e | x_1 = h) = \text{softmax}(o_1) = \frac{\exp(2.2)}{\exp(1.0) + \dots + \exp(4.1)}$$

$$o_1 = W_{hy} h_1$$

$$h_1 = \tanh(W_{hh} h_0 + W_{xh} x_1)$$

Example: Character RNN

- Train 3-layer RNN with 512 hidden nodes on all the works of Shakespeare.
- Then sample from the model:

KING LEAR: O, if you were a feeble sight, the courtesy of your law,
Your sight and several breath, will wear the gods With his heads,
and my hands are wonder'd at the deeds, So drop upon your
lordship's head, and your opinion Shall be against your honour.

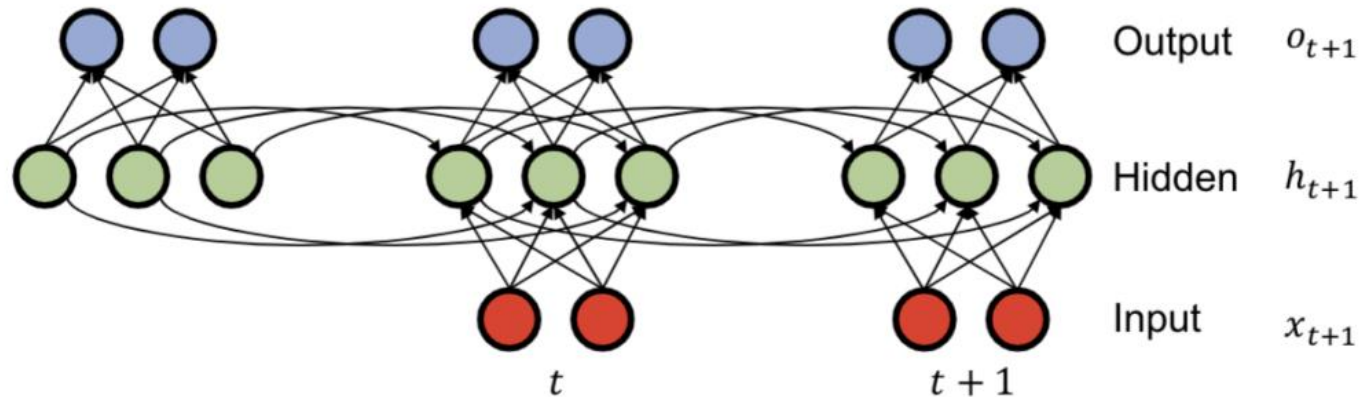
- **Note:** generation happens **character by character**. Needs to learn valid words, grammar, punctuation, etc.

Example: Character RNN

- Train on data set of baby names. Then sample from the model:

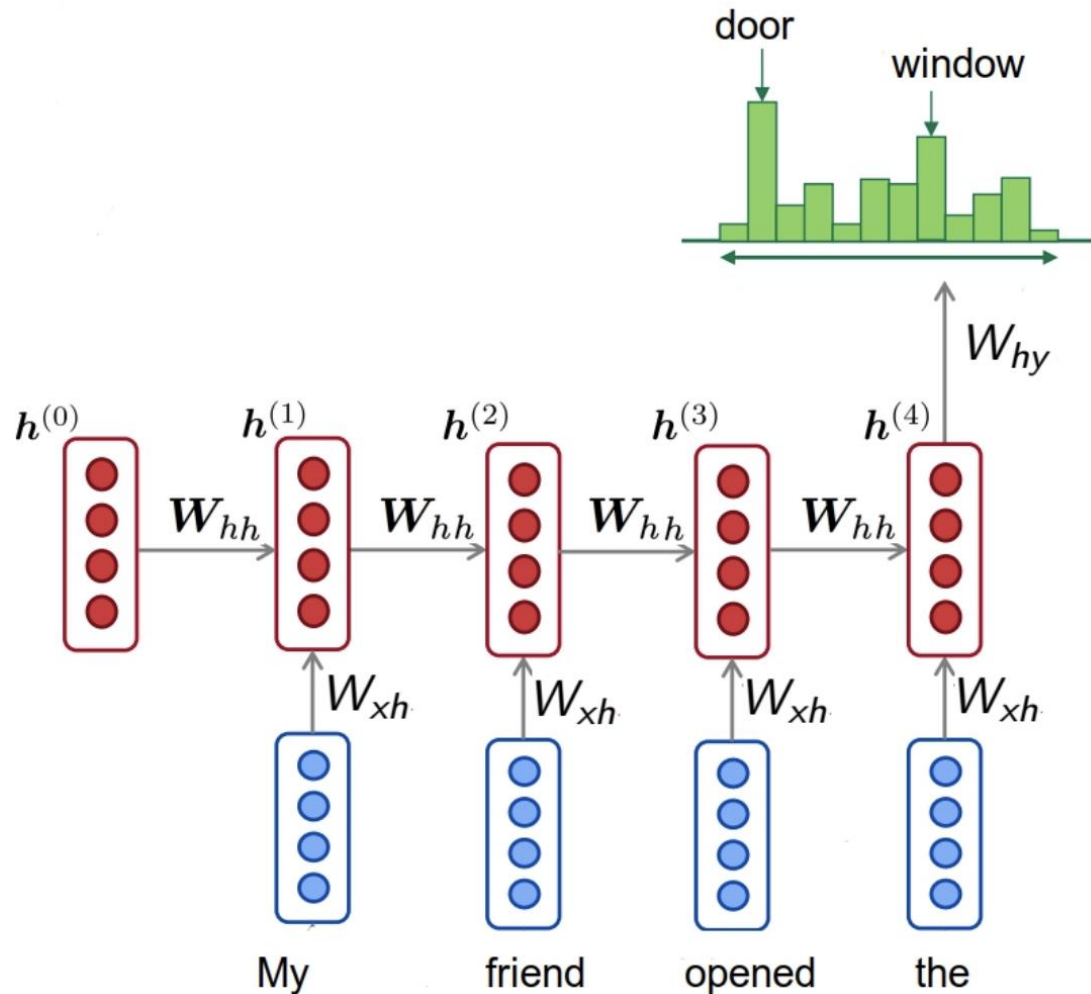
Rudi Levette Berice Lussa Hany Mareanne Chrestina Carissy Marylen Hammine
Janye Marlise Jacacrie Hendred Romand Charienna Nenotto Ette Dorane
Wallen Marly Darine Salina Elvyn Ersia Maralena Minoria Ellia Charmin Antley
Nerille Chelon Walmor Evena Jeryly Stachon Charisa Allisa Anatha Cathanie
Geetra Alexie Jerin Cassen Herbett Cossie Velen Daurenge Robester Shermond
Terisa Licia Roselen Ferine Jayn Lusine Charyanne Sales Sanny Resa Wallon
Martine Merus Jelen Candica Wallin Tel Rachene Tarine Ozila Ketia Shanne
Arnande Karella Roselina Alessia Chasty Deland Berther Geamar Jackein
Mellisand Sagdy Nenc Lessie Rasemy Guen Gavi Milea Anneda Margoris Janin
Rodelin Zeanna Elyne Janah Ferzina Susta Pey Castina

RNN: Recurrent Neural Nets



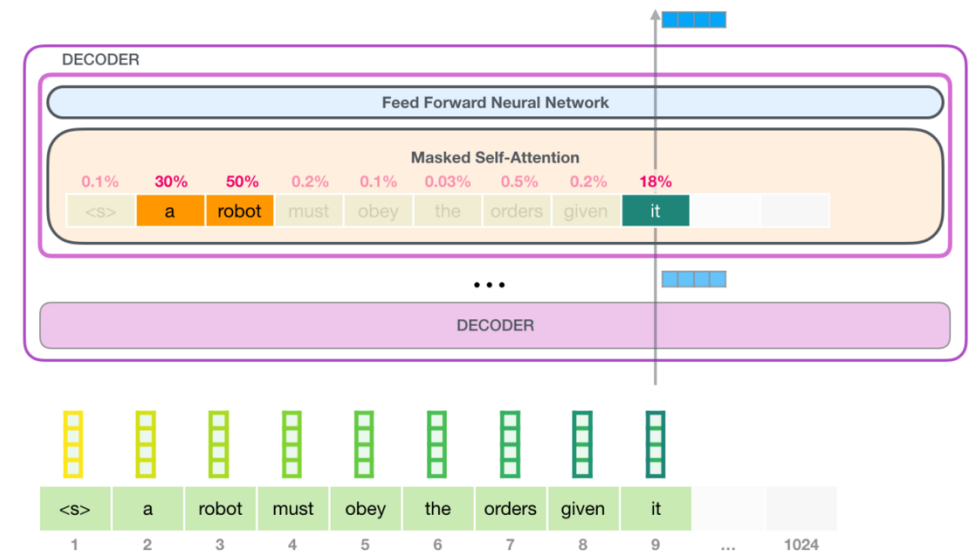
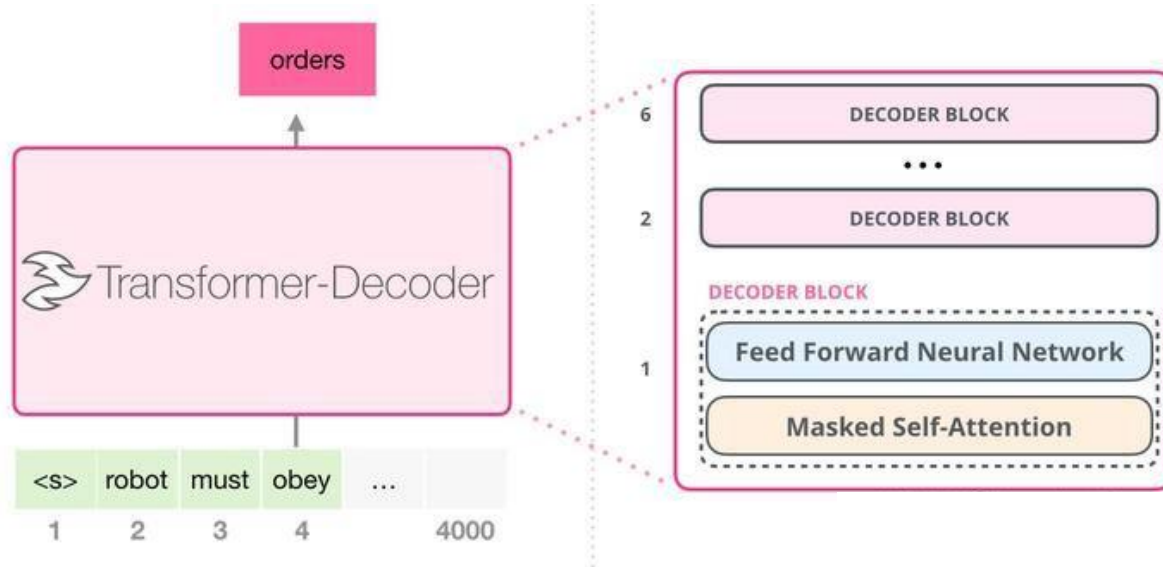
- Pros:
 - Can be applied to sequences of arbitrary length.
 - Very general: For every computable function, there exists a finite RNN that can compute it
- Cons:
 - Sequential likelihood evaluation (very slow for training)
 - Sequential generation (unavoidable in an autoregressive model)

Issues with RNNs



- A single hidden vector needs to summarize all the (growing) history. For example, $h^{(4)}$ needs to summarize the meaning of “My friend opened the”.
- Sequential evaluation, cannot be parallelized.
- Exploding/vanishing gradients when accessing information from many steps back.

Generative Transformers



Current state-of-the-art LLM: Transformer

- Attention mechanisms to adaptively focus only on relevant context.
- Avoid recursive computation. Use only self-attention to enable parallelization.
- Needs masked self-attention to preserve autoregressive structure
- Demo: <https://huggingface.co/Qwen/Qwen3.8-27B>
- Demo: <https://huggingface.co/meta-llama/Llama-4-Scout-17B-16E-Instruct>

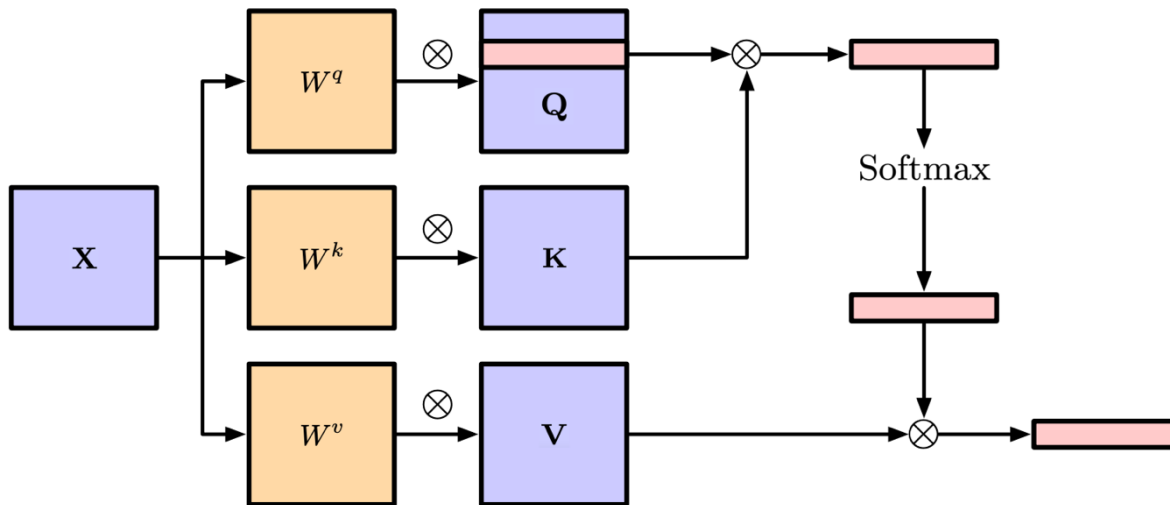
Self-Attention Mechanism

Figure modified from <https://zhuanlan.zhihu.com/p/441240252>

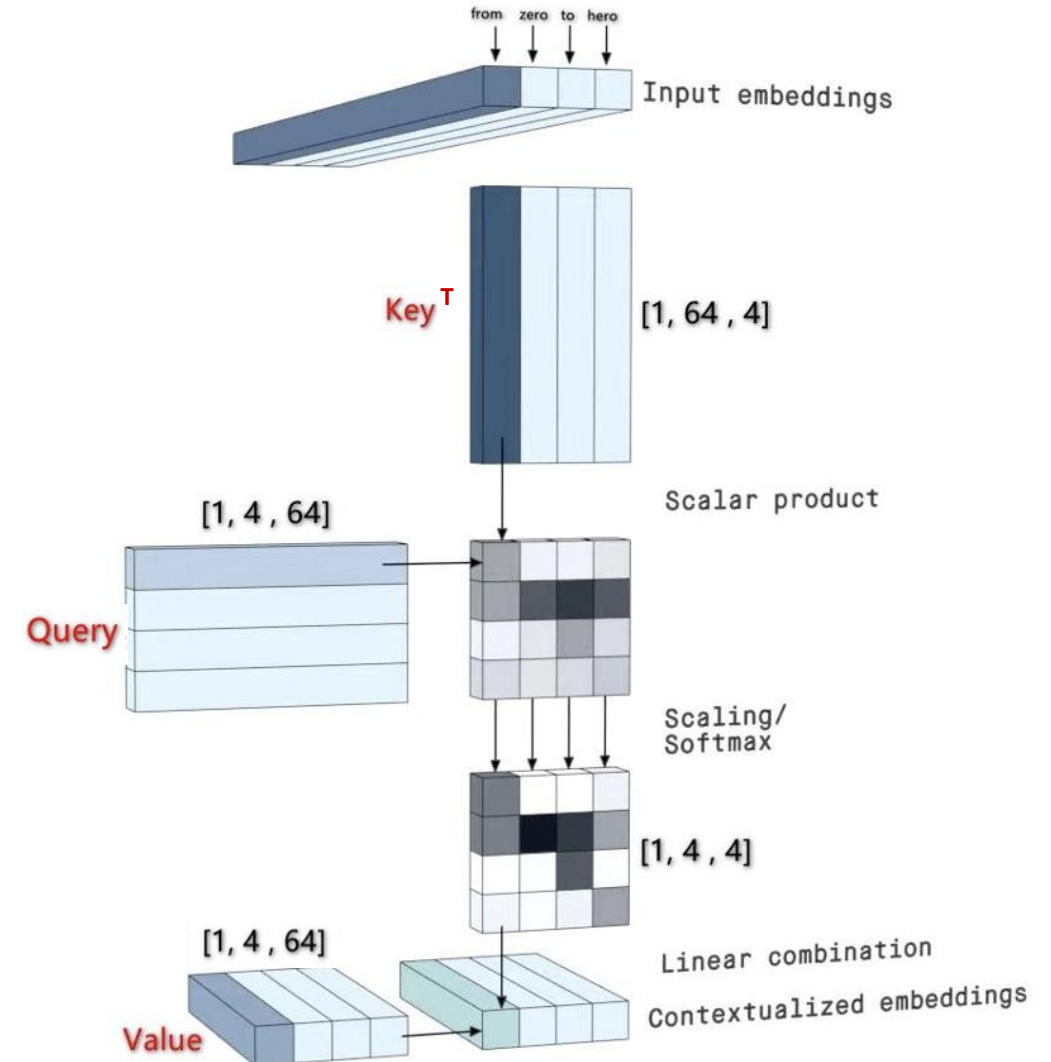
$$\mathbf{Q} = \mathbf{XW}^{(q)} \in \mathbb{R}^{N \times D_k}$$

$$\mathbf{K} = \mathbf{XW}^{(k)} \in \mathbb{R}^{N \times D_k}$$

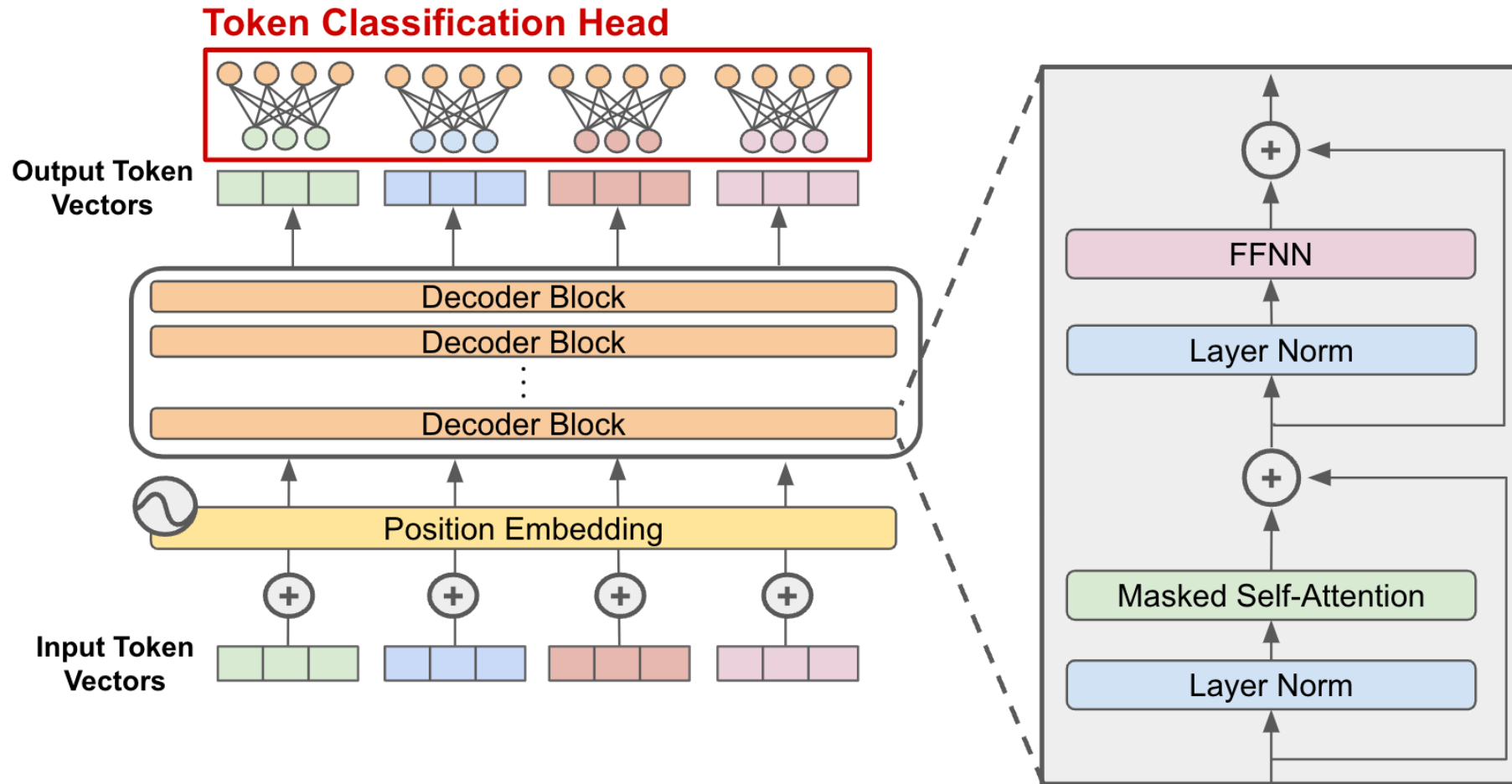
$$\mathbf{V} = \mathbf{XW}^{(v)} \in \mathbb{R}^{N \times D_v}$$



$$\mathbf{Y} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \equiv \text{Softmax} \left[\frac{\mathbf{QK}^T}{\sqrt{D_k}} \right] \mathbf{V}$$



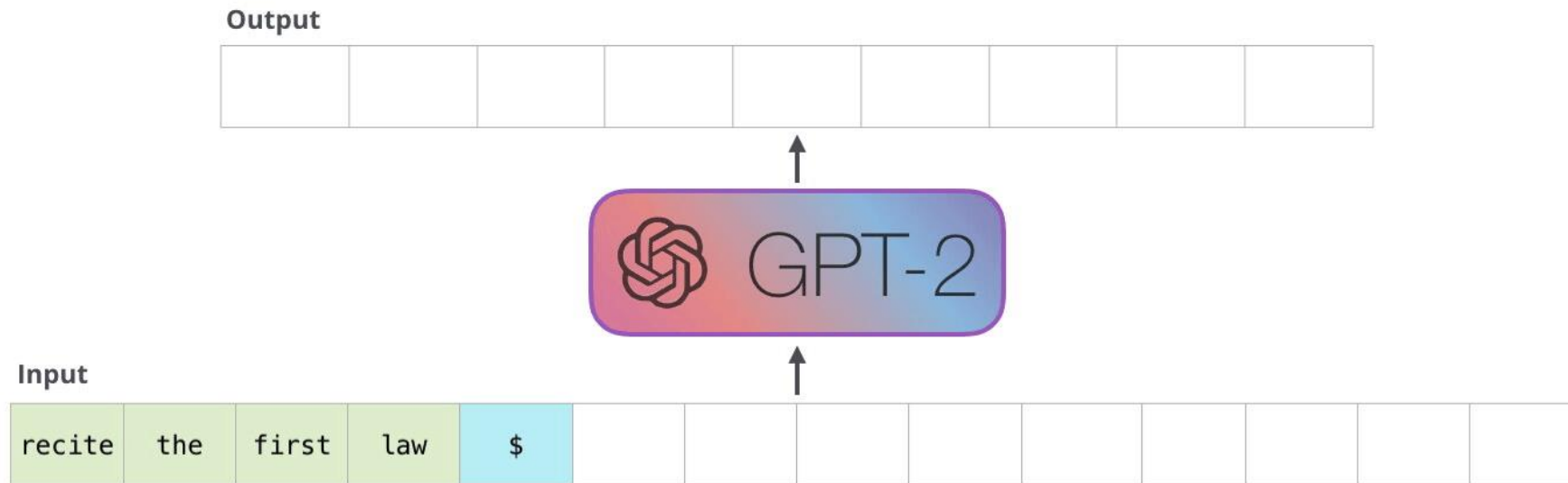
Training phase and Generation phase



$$p(x_1, x_2, x_3, x_4) \approx p(x_1)p(x_2 \mid x_1)p_{\text{Neural}}(x_3 \mid x_1, x_2)p_{\text{Neural}}(x_4 \mid x_1, x_2, x_3)$$

Next Token Prediction

Next Token Prediction (NTP) is the basic task of LLMs, where the model predicts the most likely next word or piece of text based on what has been written so far.



Word Vocabulary

Text
sample:

LLMs learn to predict one word at a time

LLMs learn to predict one word at a time

LLMs learn to predict one word at a time

LLMs learn to predict one word at a time

LLMs learn to predict one word at a time

LLMs learn to predict one word at a time

LLMs learn to predict one word at a time

LLMs learn to predict one word at a time

The LLM
can't access
words past
the target

Target to
predict

Input the
LLM
receives

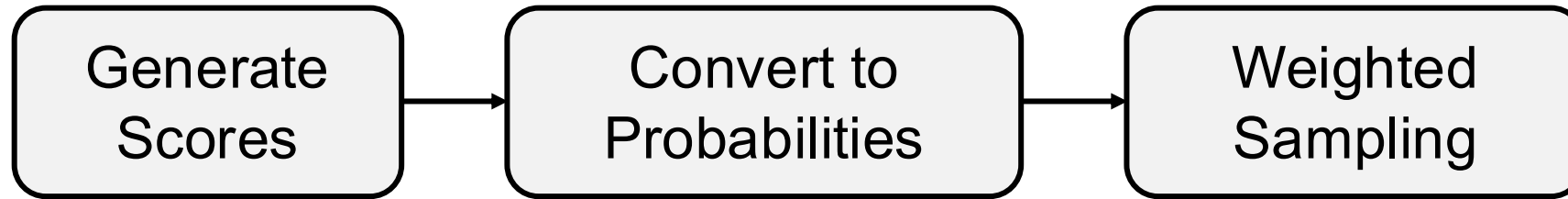
Word Vocabulary

Tokens	Characters
53	261

Lexical tokenization is conversion of a text into (semantically or syntactically) meaningful lexical tokens belonging to categories defined by a "lexer" program. In case of a natural language, those categories include nouns, verbs, adjectives, punctuations etc.

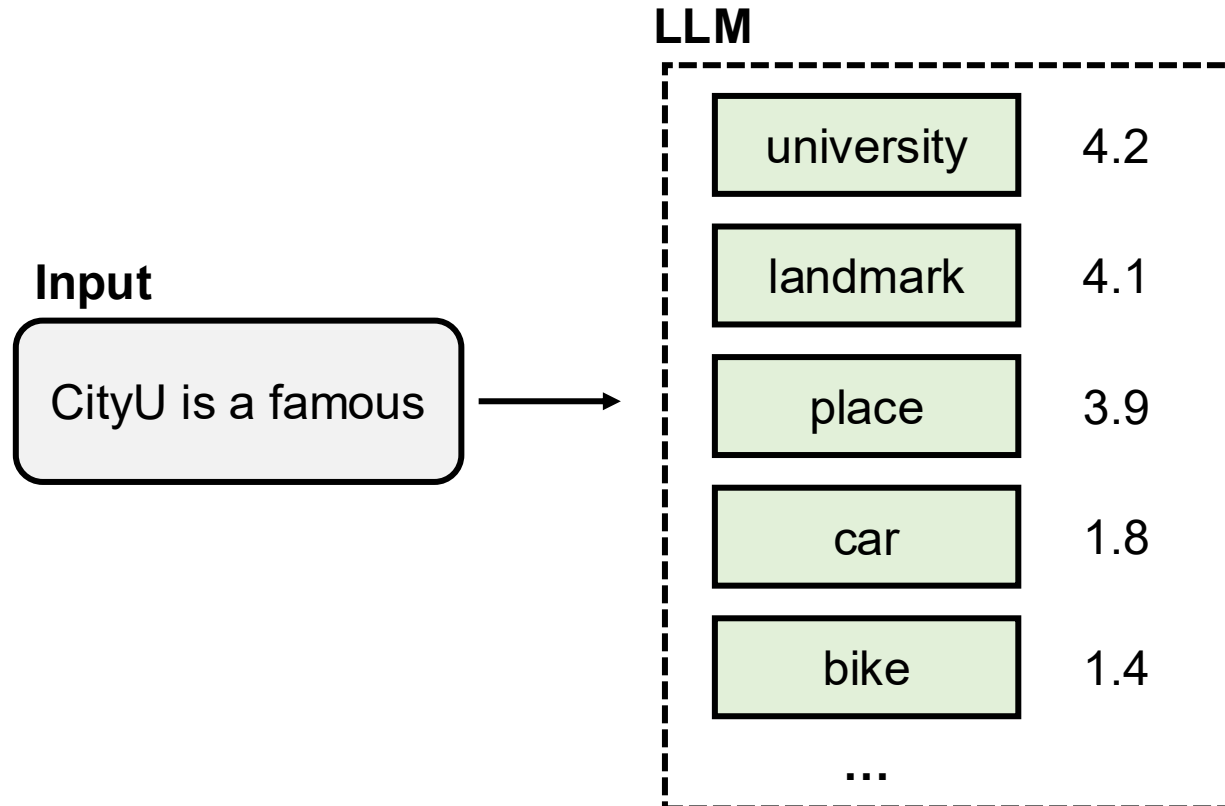
TEXT TOKEN IDS

Three Steps of NTP

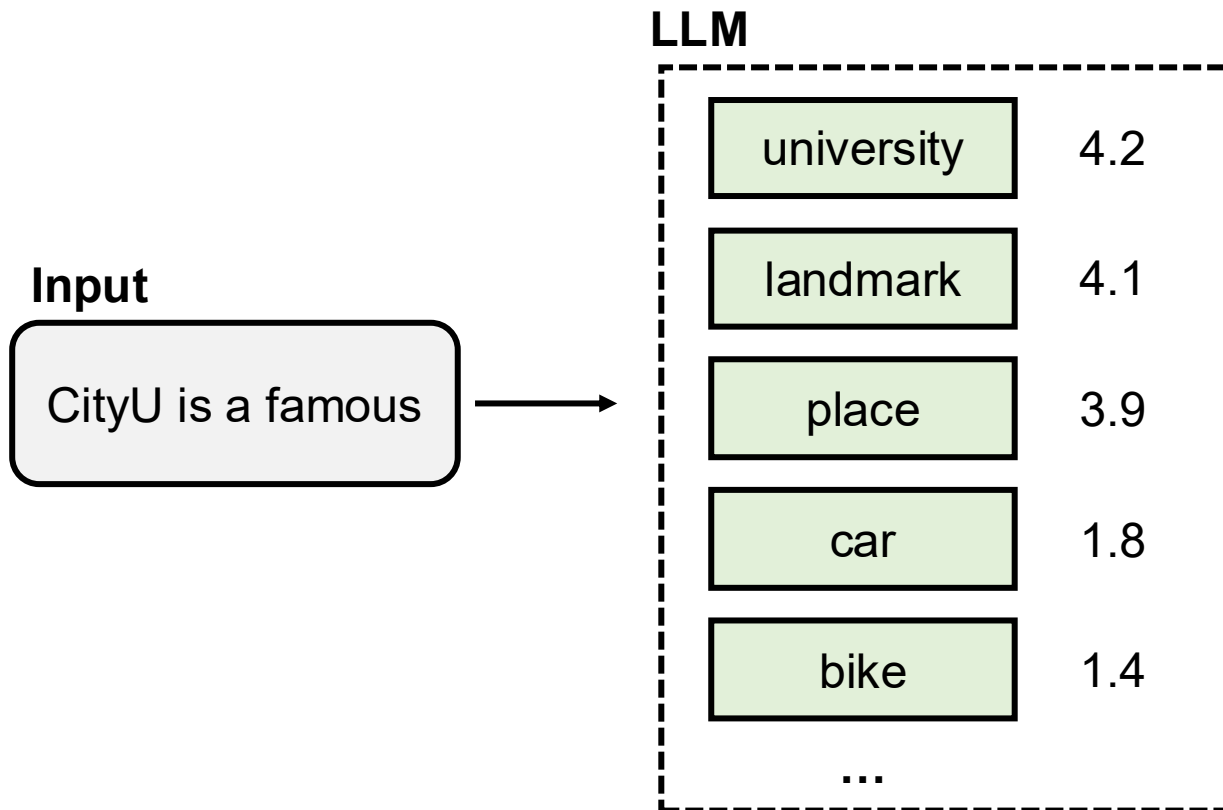


Step 1. Generate Scores (logits)

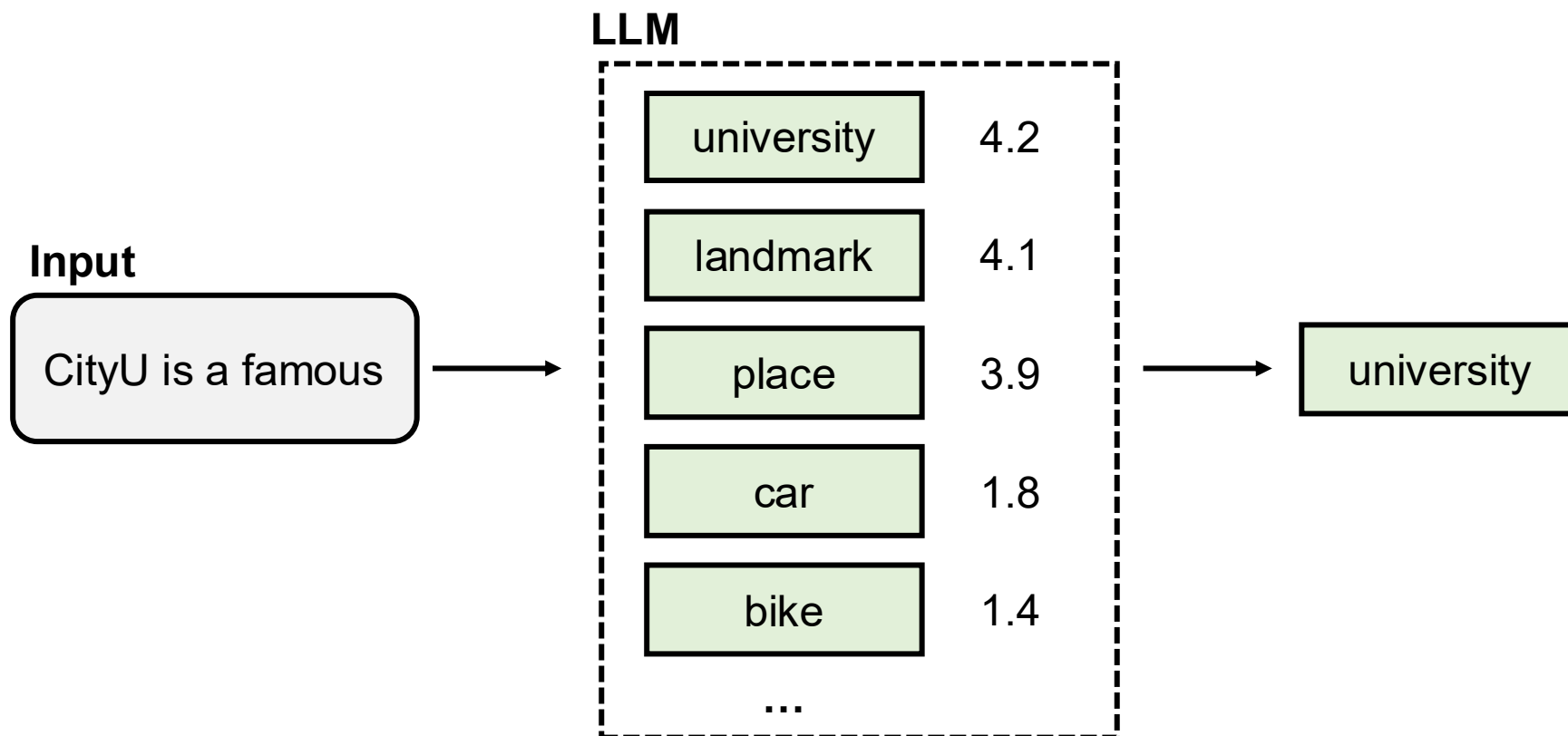
The model reads the input and calculates a score for every single possible next word in the vocabulary (tens of thousands of words).



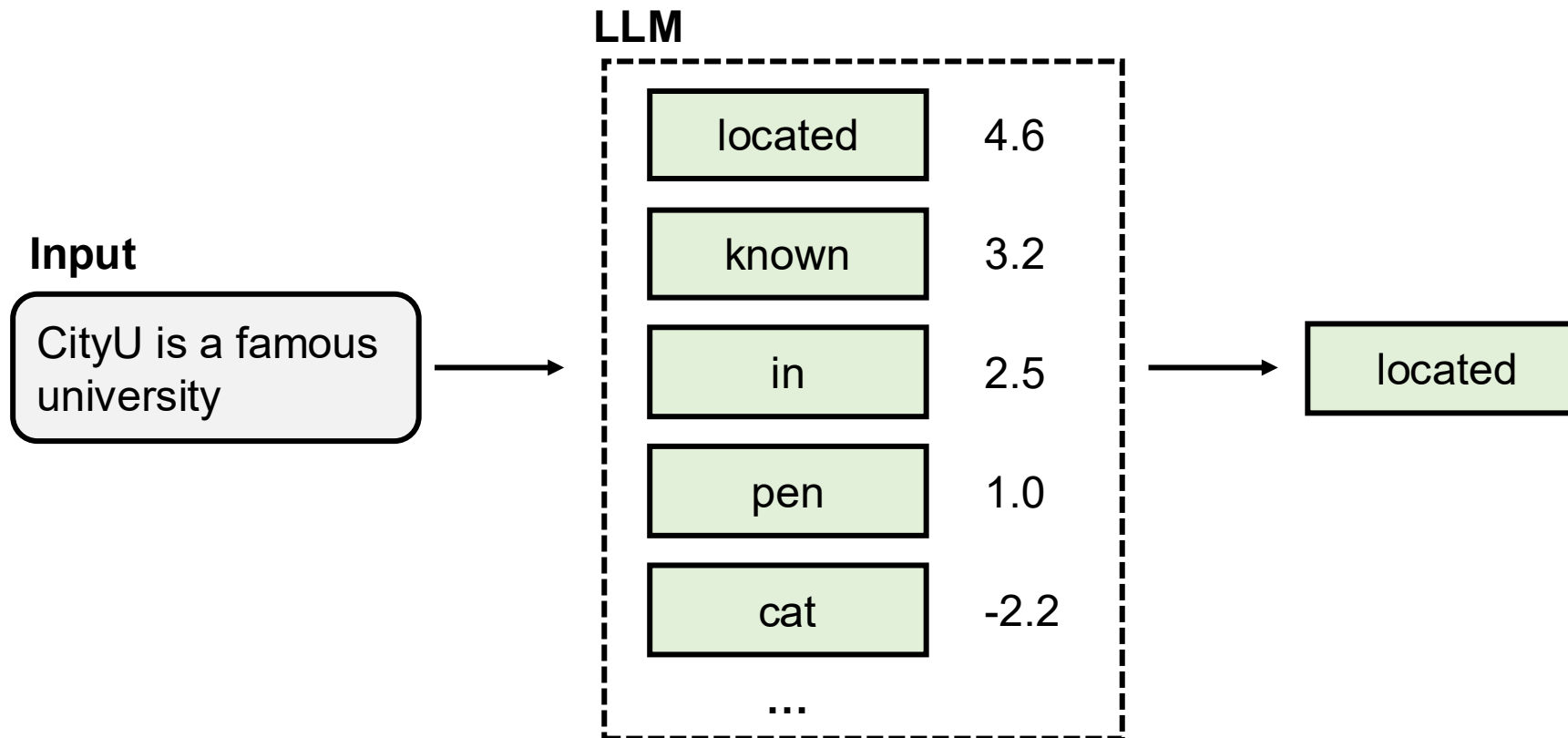
Which Word is Next?



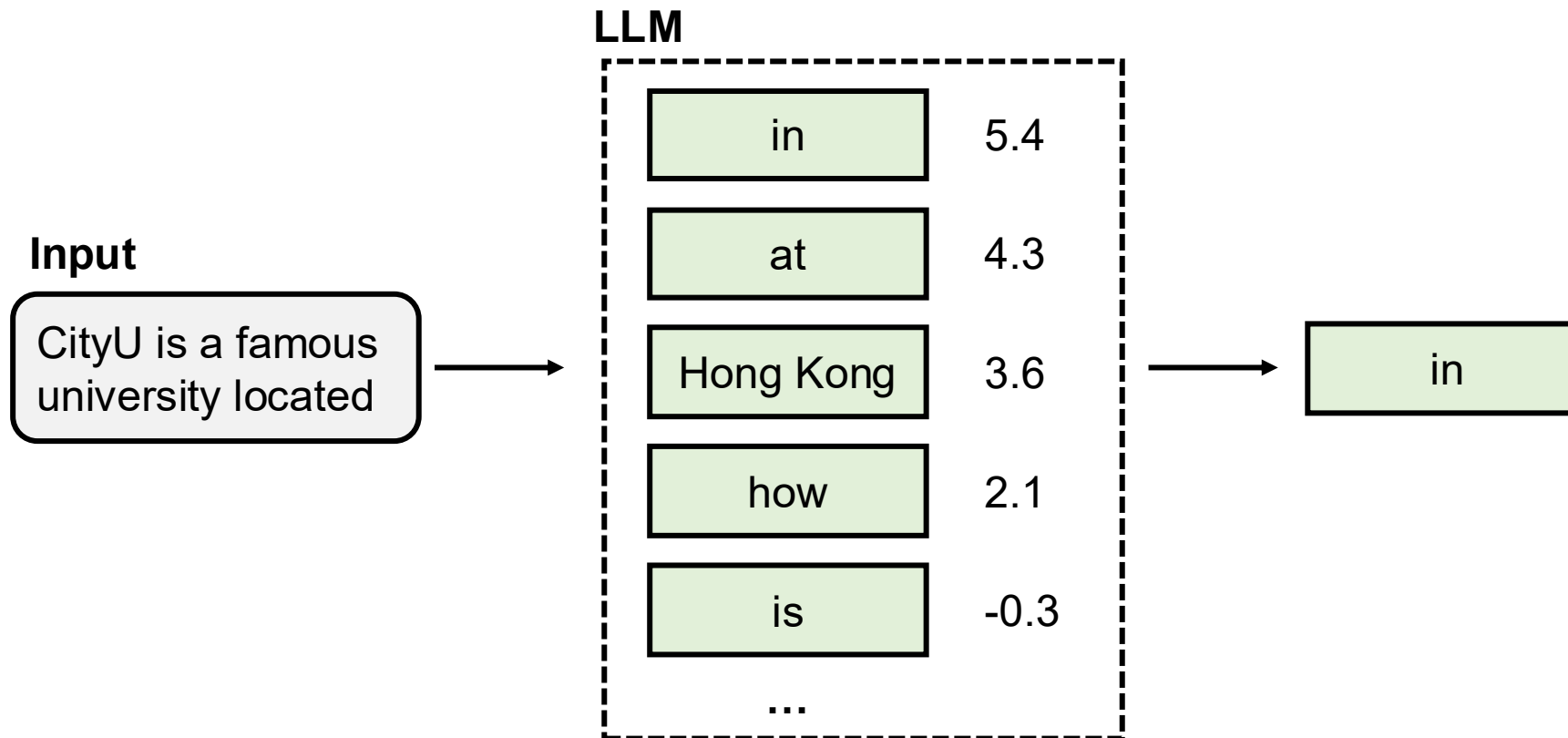
Select Word with the Highest Score



Continuing the Cycle

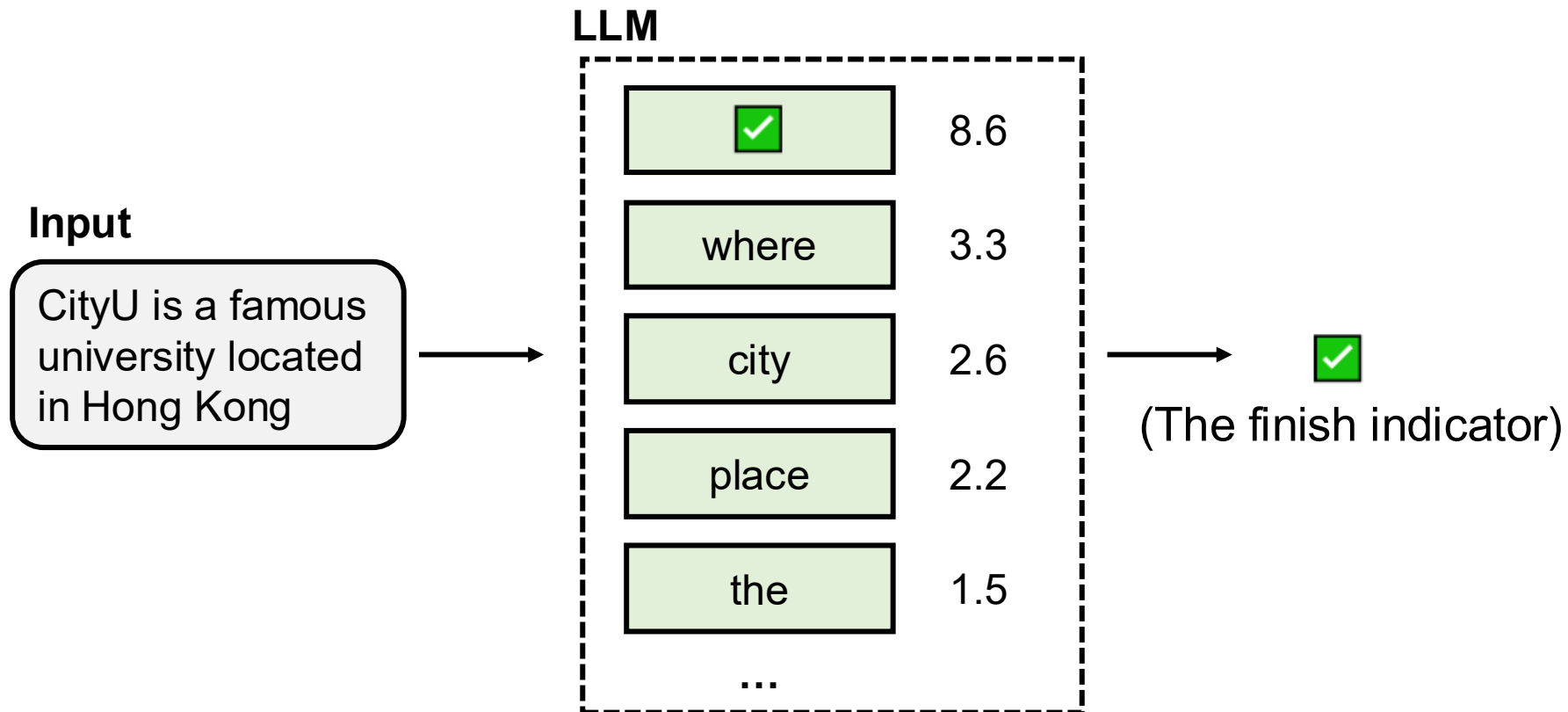


Continuing the Cycle



The model repeats this process until generation is complete.

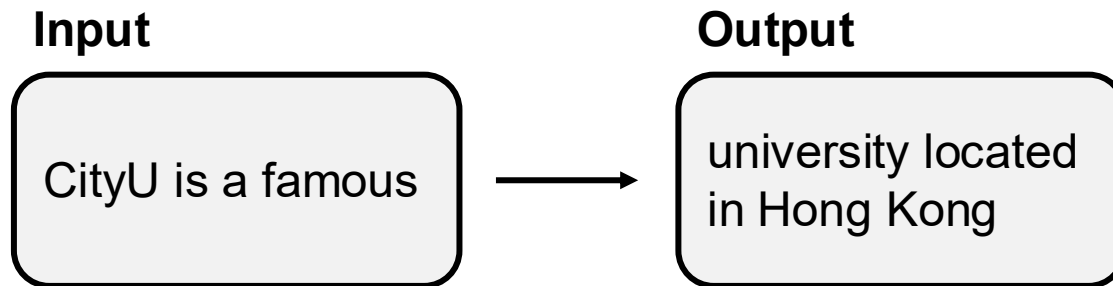
Continuing the Cycle



The model repeats this process until generation is complete.

The Problem

The model always produces the same output for the same input.



Step 2. Convert to Probabilities

The generated score can be converted to probabilities by **Softmax** function.

LLM

university	4.2	z_1
landmark	4.1	z_2
place	3.9	z_3
car	1.8	z_4
bike	1.4	z_5

$$\text{Softmax}(z_1) \approx 36\%$$

$$\text{Softmax}(z_2) \approx 32\%$$

$$\text{Softmax}(z_3) \approx 26\%$$

$$\text{Softmax}(z_4) \approx 3\%$$

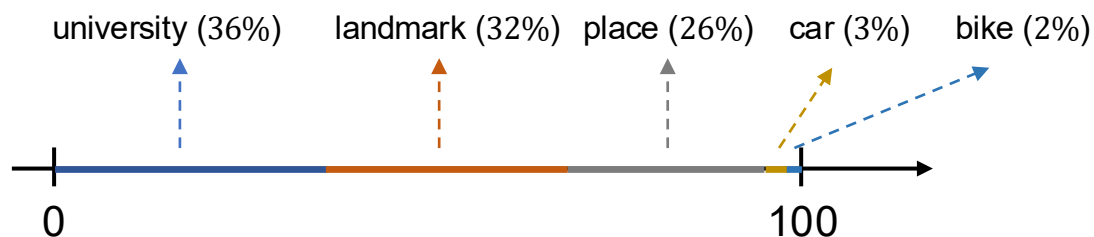
$$\text{Softmax}(z_5) \approx 2\%$$

Step 3. Weighted Sampling

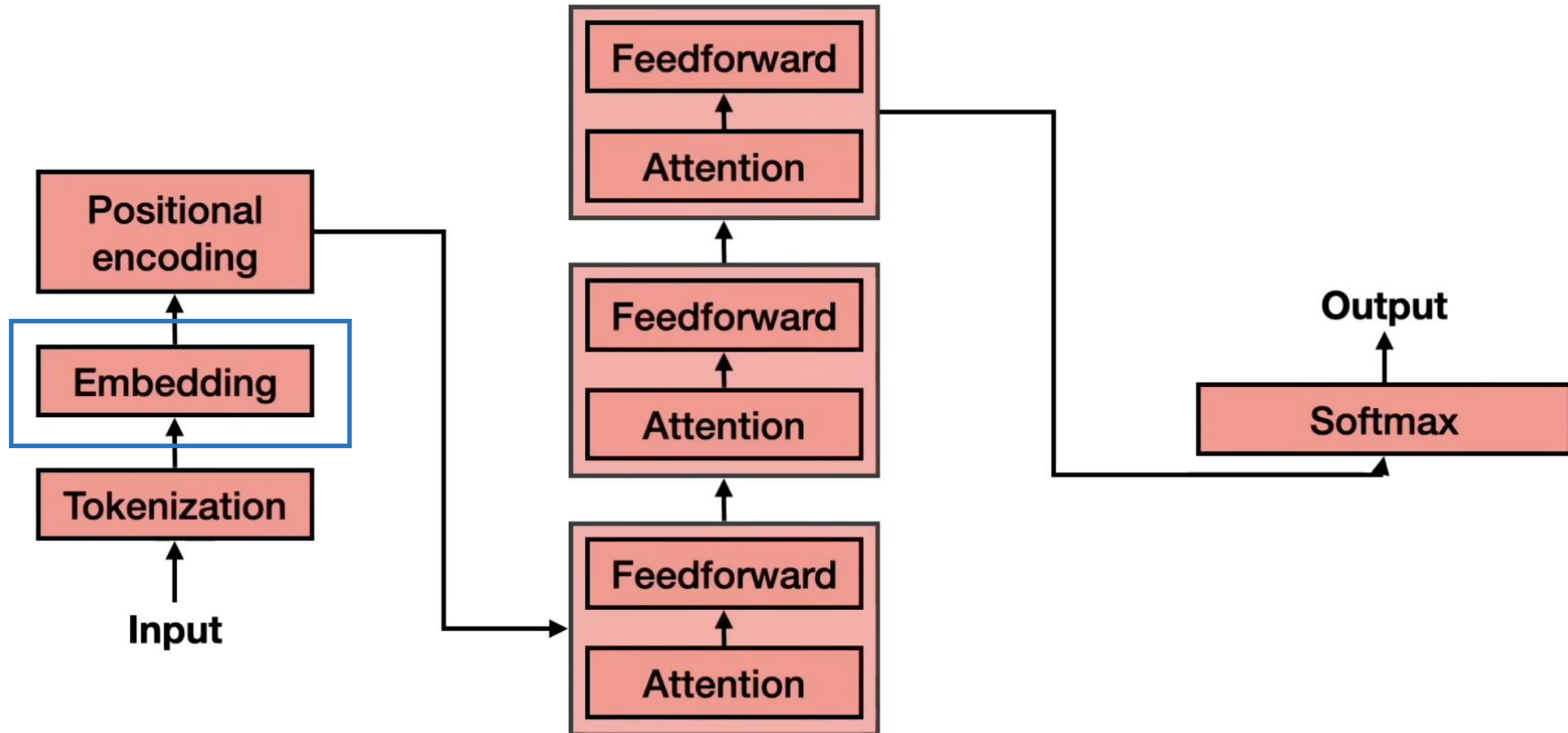
The model uses a strategy called “Weighted Sampling” to predict the output based on their probabilities.

LLM

university	36%
landmark	32%
place	26%
car	3%
bike	2%

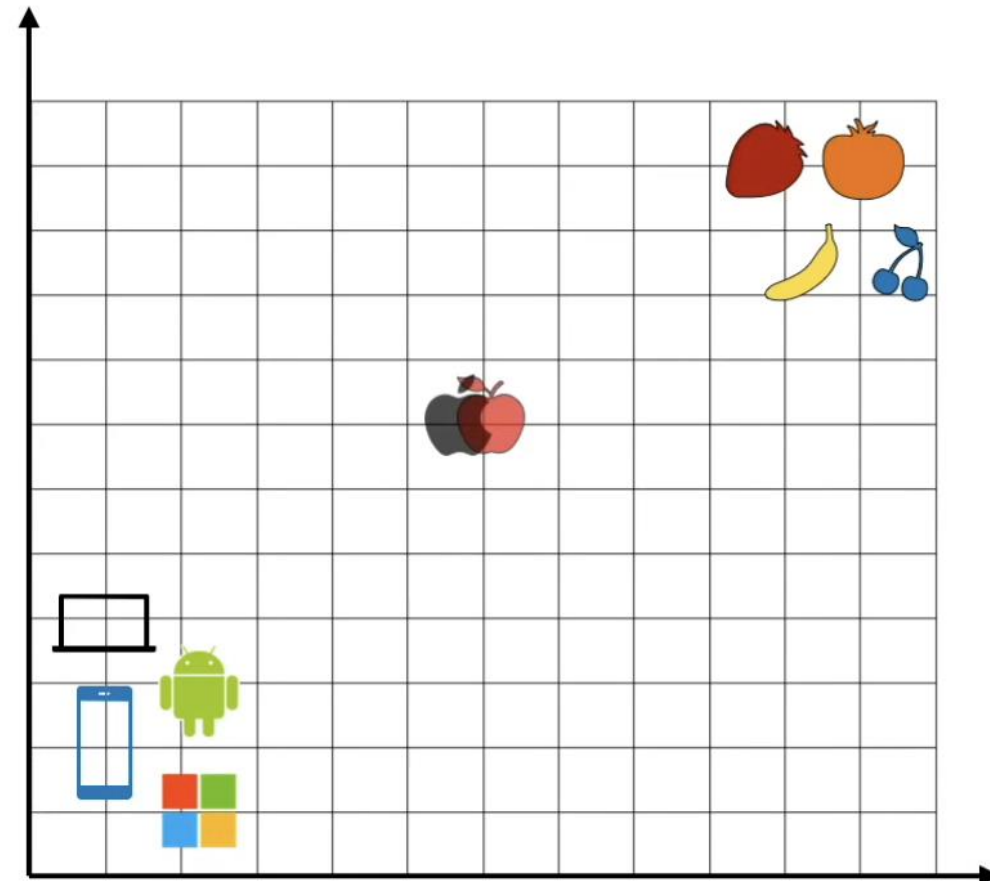
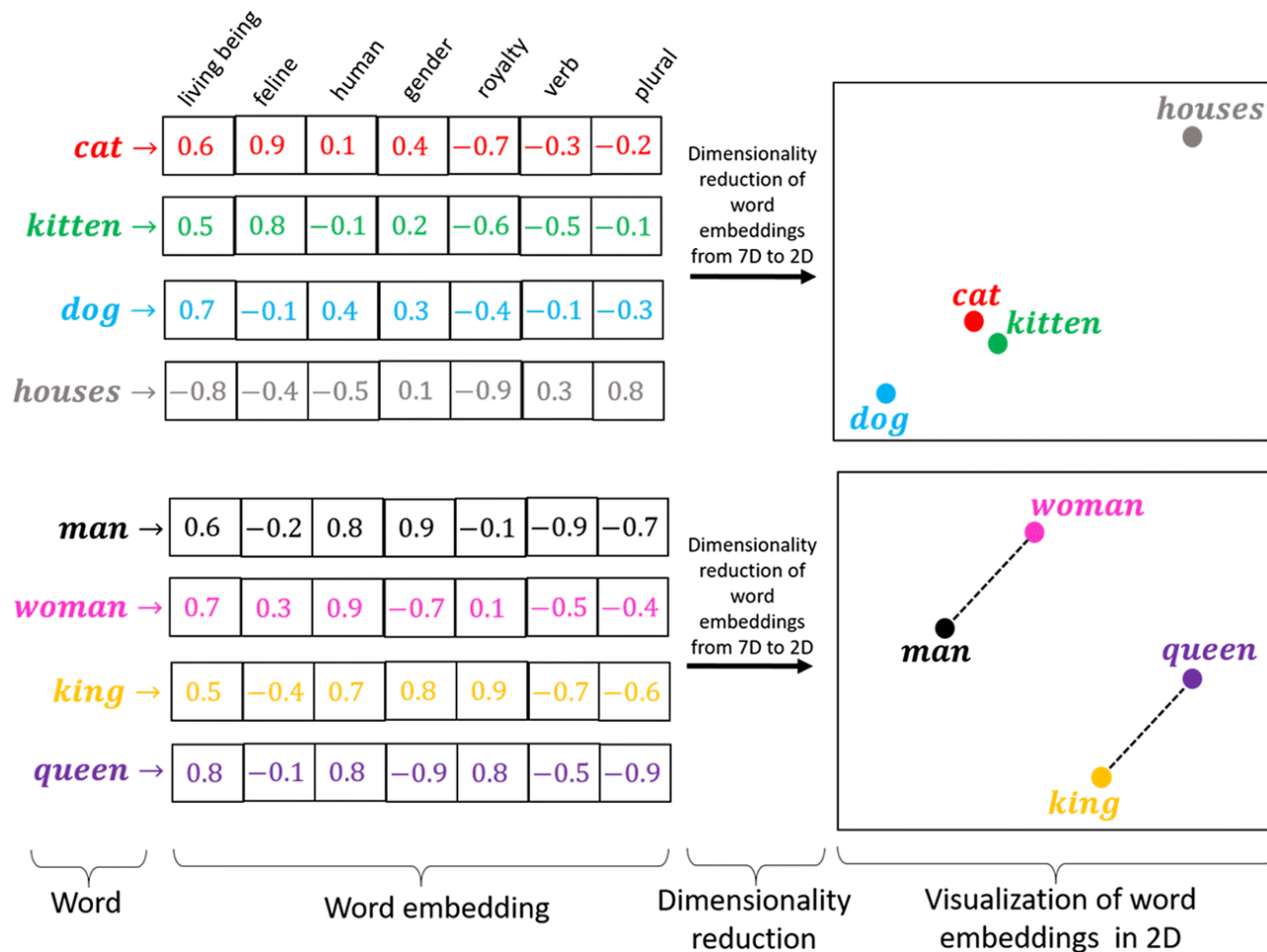


Overall Arch of GPT/Llama/DeepSeek

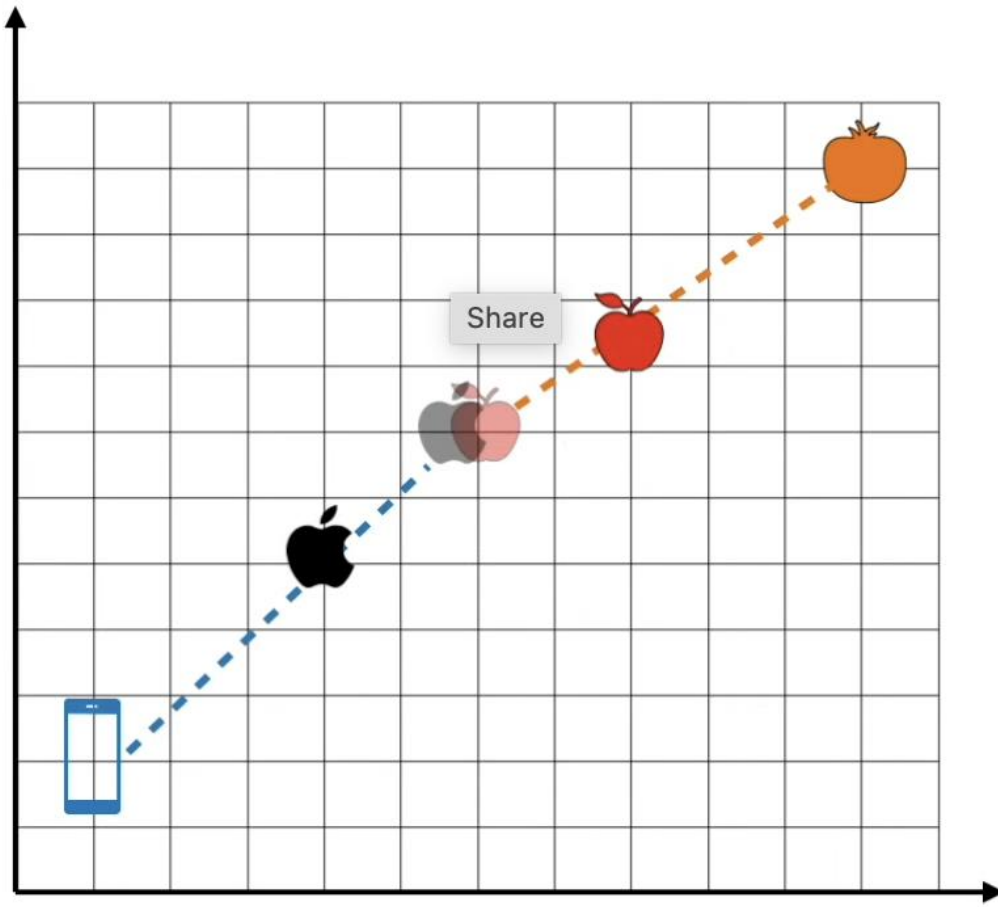


What is Embedding

What is Embedding



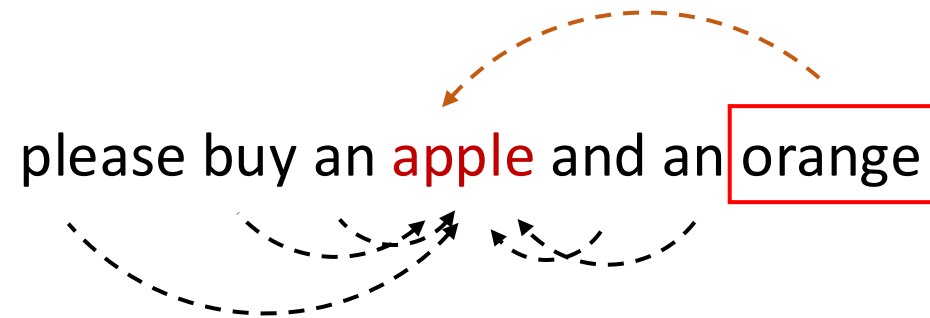
Words pulling words



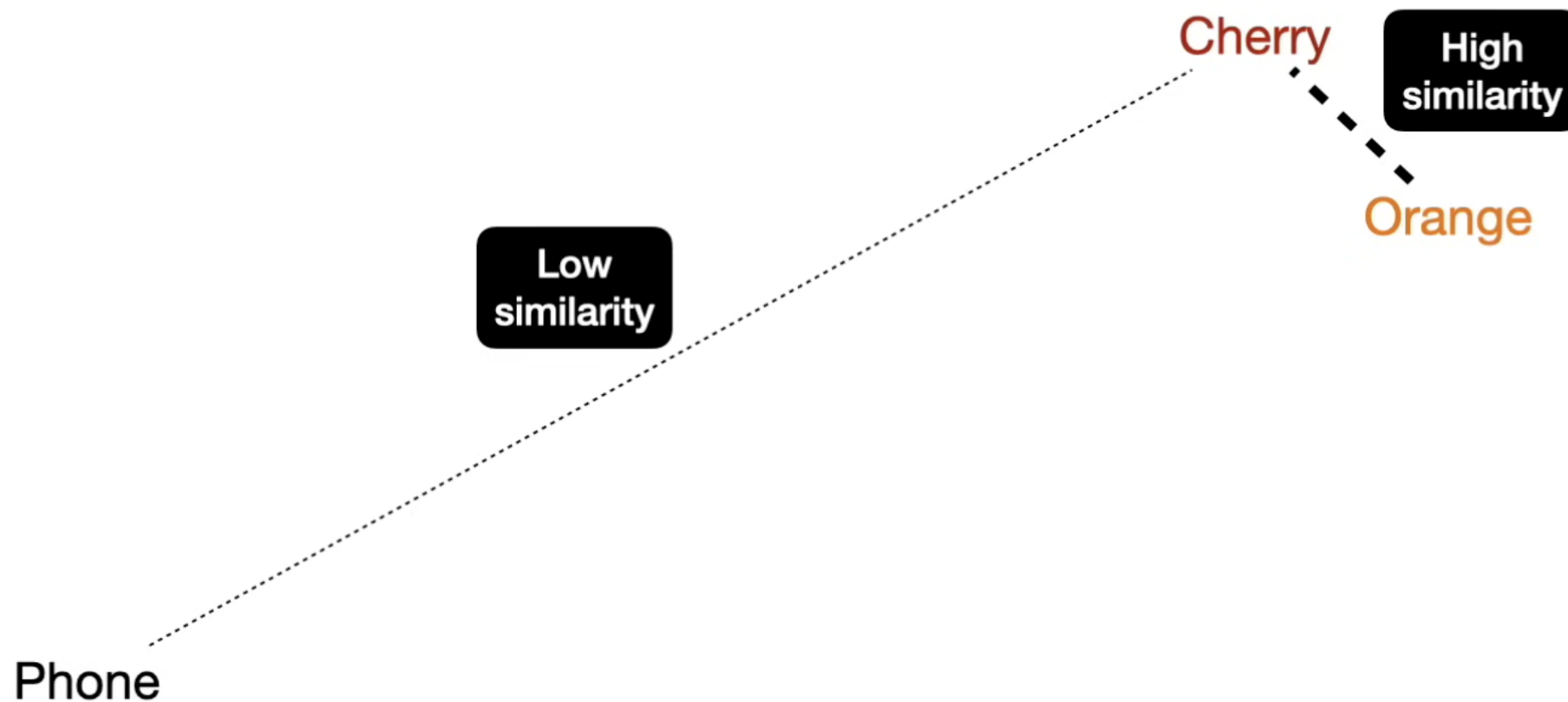
please buy an **apple** and an **orange**

apple unveiled the new **phone**

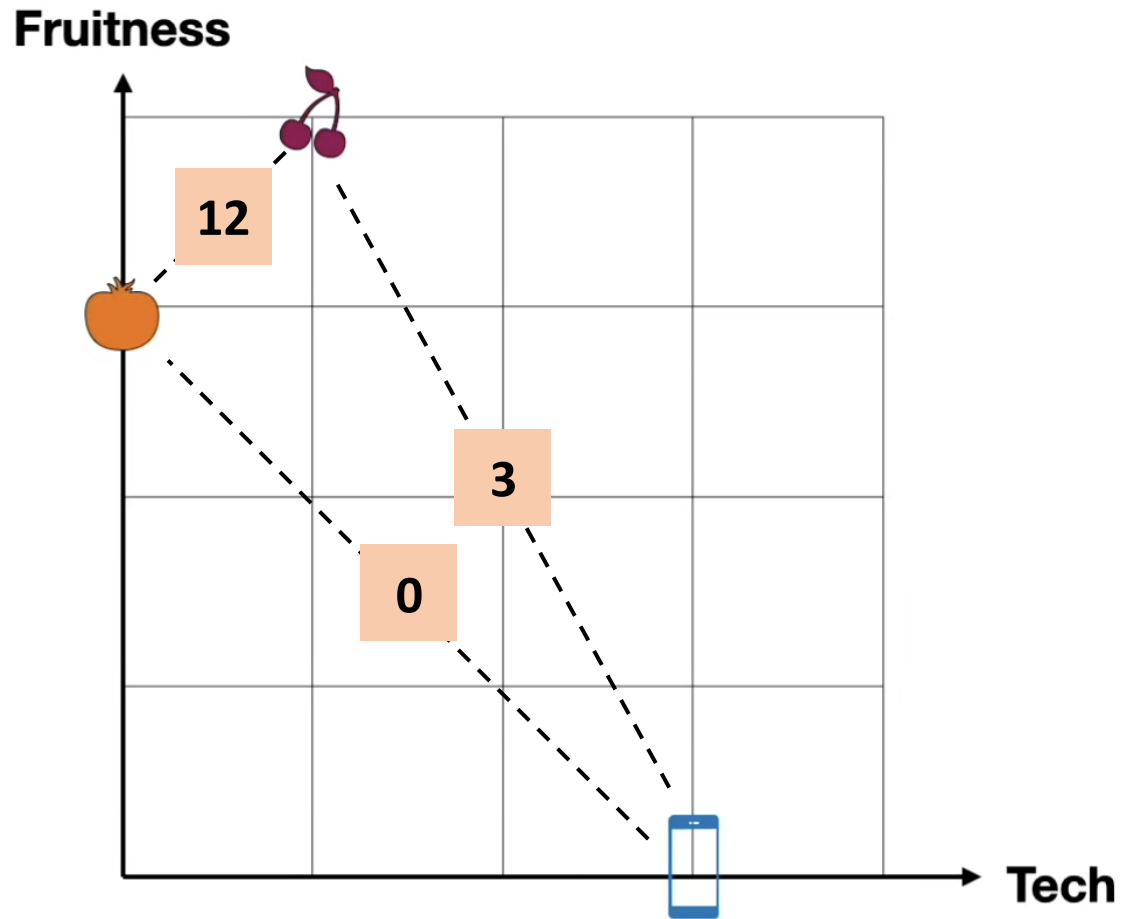
All words pull each other



Similarity

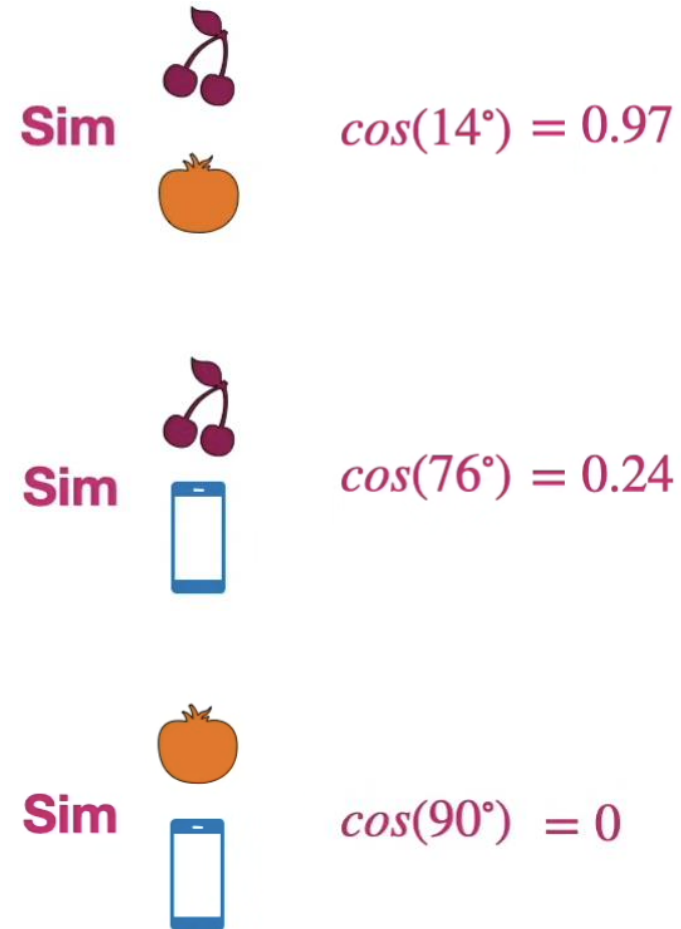
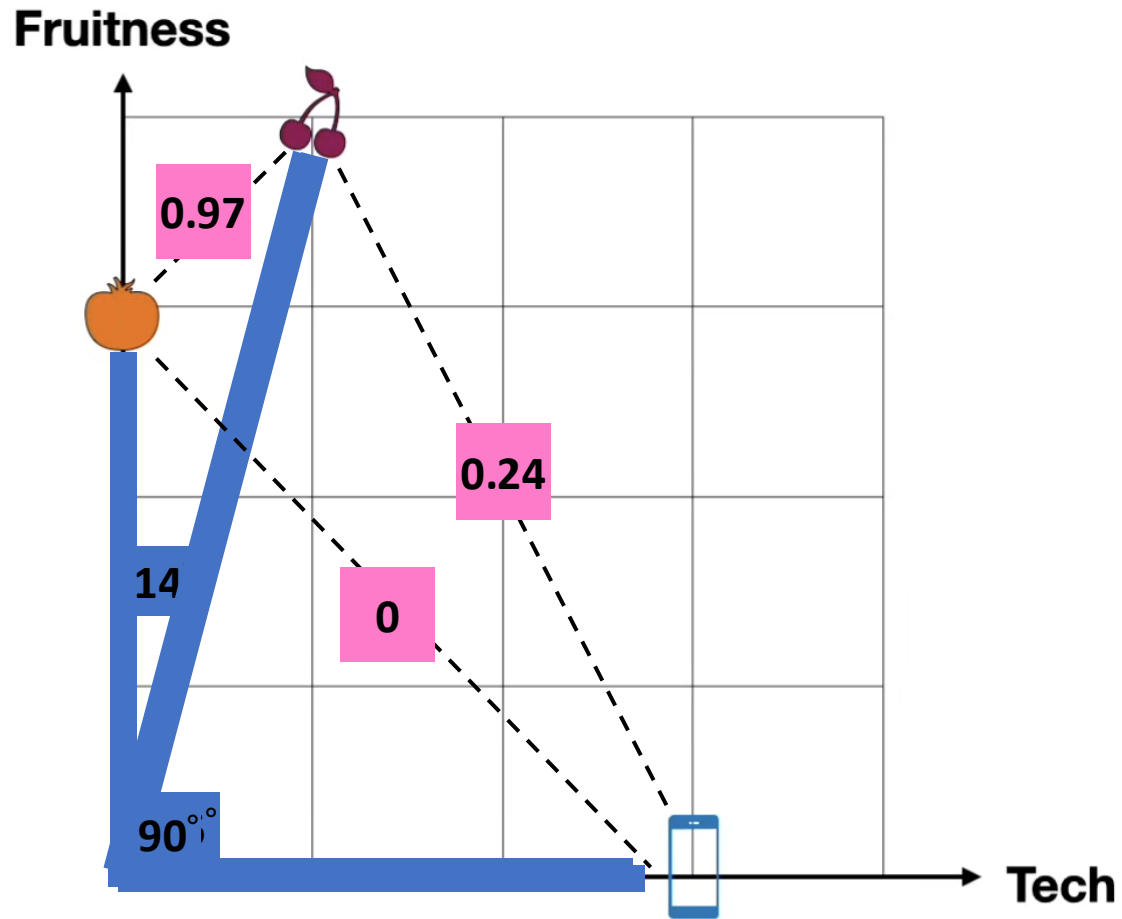


Measure 1: Dot Product

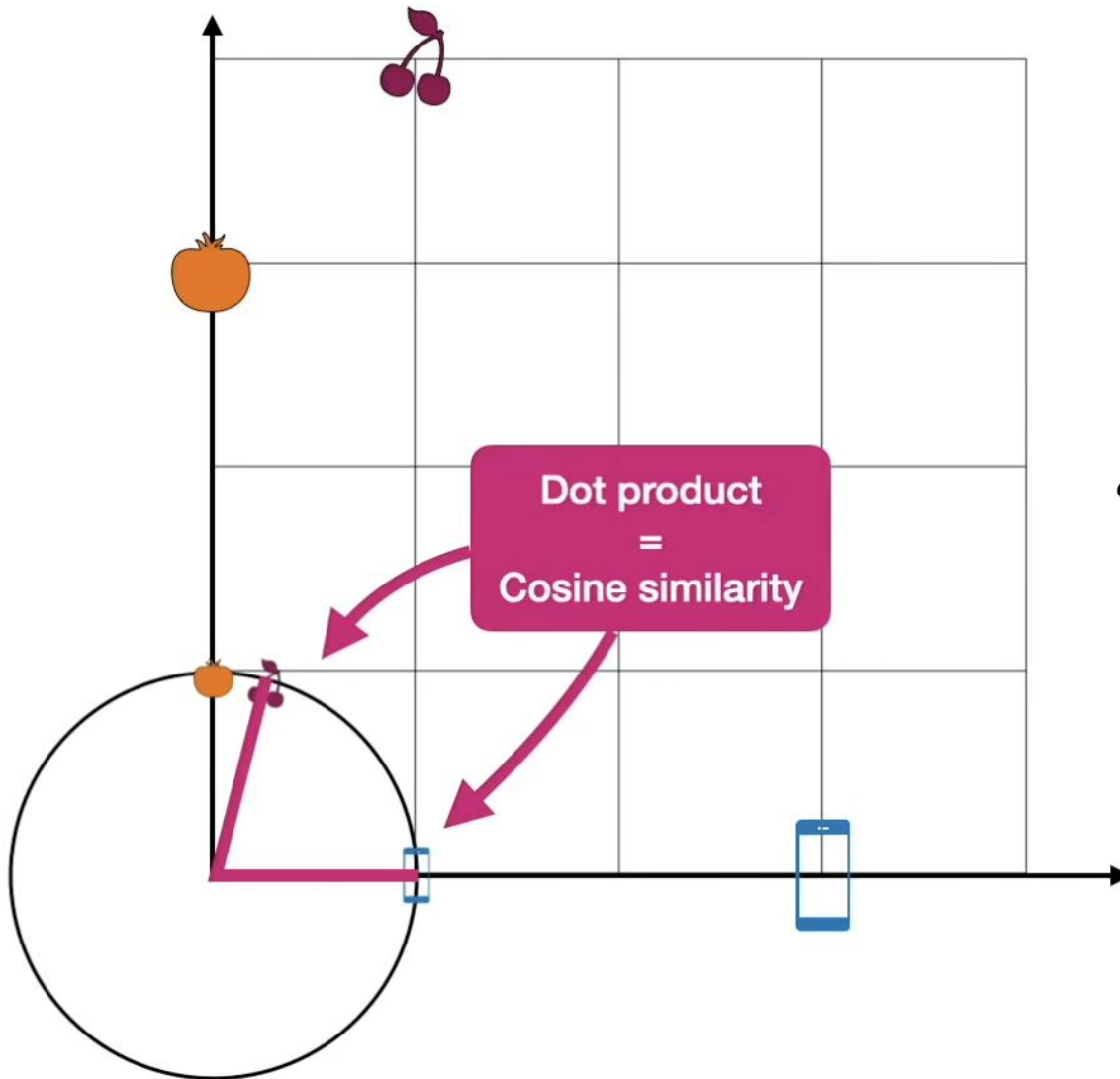


		Tech	Fruitiness	
Sim		1	4	$1 \cdot 0 + 4 \cdot 3 = 12$
		0	3	
Sim		1	4	$1 \cdot 3 + 4 \cdot 0 = 3$
		3	0	
Sim		0	3	$0 \cdot 3 + 3 \cdot 0 = 0$
		3	0	

Measure 2: Cosine Similarity



Dot Product and Cos Similarity



$$\text{cosine similarity} = S_C(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}},$$

Measure 3: Scaled Dot Product

Sim 

1	4
---	---

Sim 

0	3
---	---

$$1 \cdot 0 + 4 \cdot 3 = 12 \longrightarrow \frac{12}{\sqrt{2}} = 8.49$$

Sim 

1	4
---	---

Sim 

3	0
---	---

$$1 \cdot 3 + 4 \cdot 0 = 3 \longrightarrow \frac{3}{\sqrt{2}} = 2.12$$

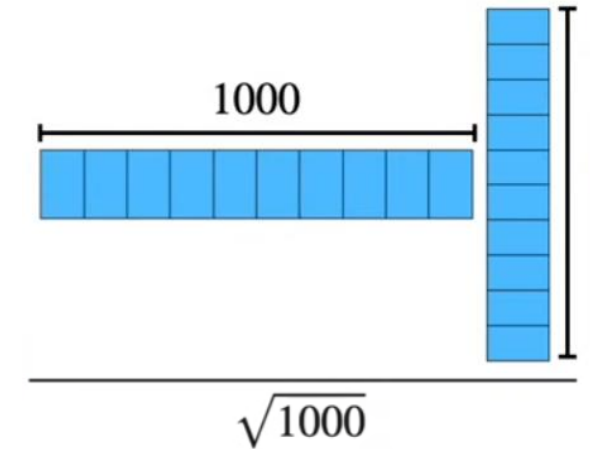
Sim 

0	3
---	---

Sim 

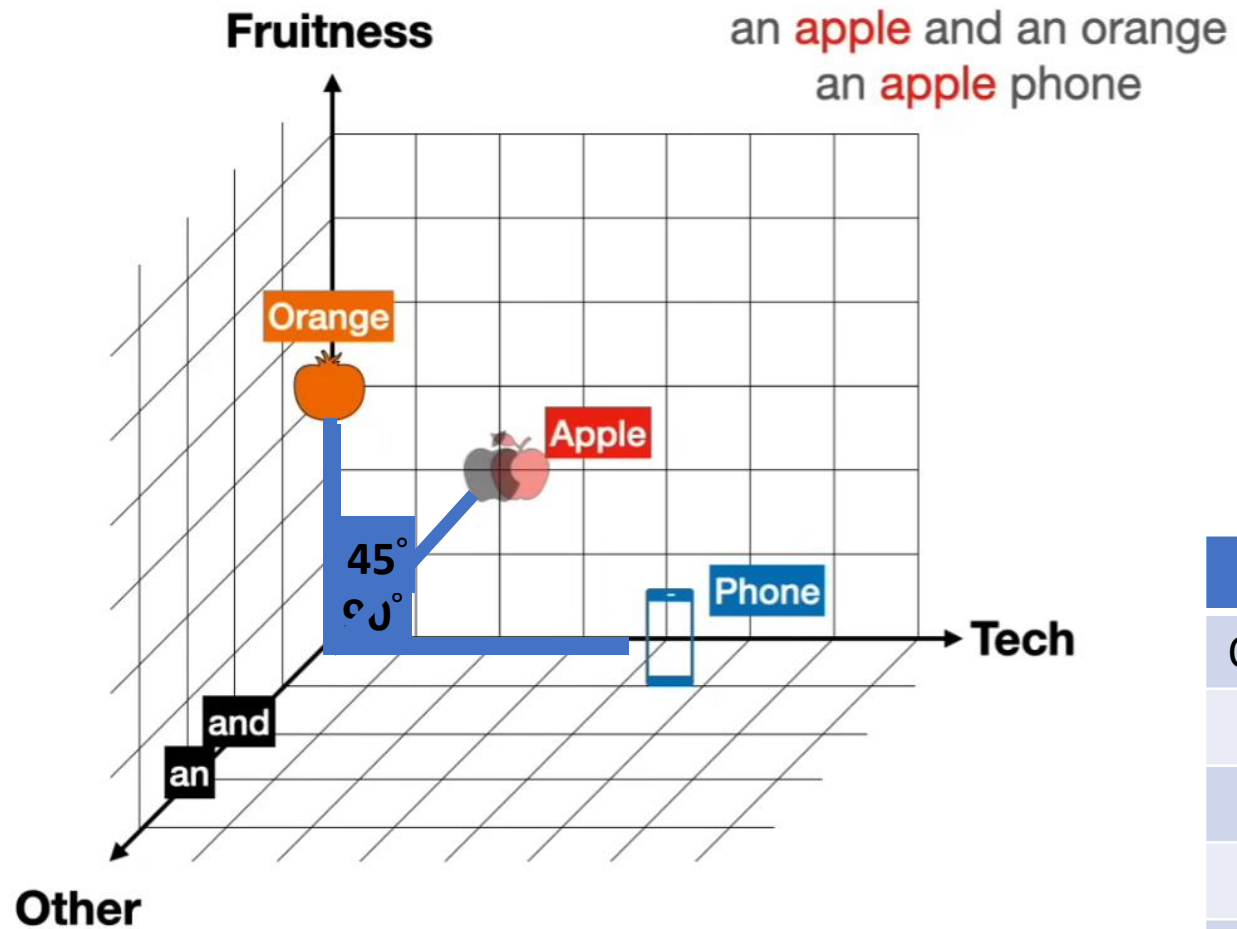
3	0
---	---

$$0 \cdot 3 + 3 \cdot 0 = 0 \longrightarrow \frac{0}{\sqrt{2}} = 0$$



Attention Mechanism

Cosine Similarity

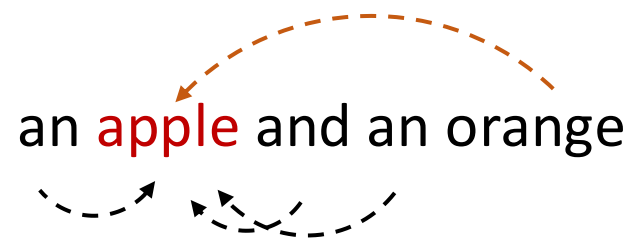


	Tech	Fruitness	Other
Orange	0	3	0
Phone	4	0	0
Apple	2	2	0
And	0	0	2
An	0	0	3

	Orange	Phone	Apple	And	An
Orange	1	0	0.71	0	0
Phone	0	1	0.71	0	0
Apple	0.71	0.71	1	0	0
And	0	0	0	1	1
An	0	0	0	1	1

Word math

an **apple** and an orange



	Orange	Apple	And	An
Orange	1	0.71	0	0
Apple	0.71	1	0	0
And	0	0	1	1
An	0	0	1	1

Orange $\rightarrow$ 1 **Orange** + 0.71 **Apple**

Apple $\rightarrow$ 0.71 **Orange** + 1 **Apple**

And $\rightarrow$ 1 **And** + 1 **An**

An $\rightarrow$ 1 **An** + 1 **And**

Word math

an **apple** phone

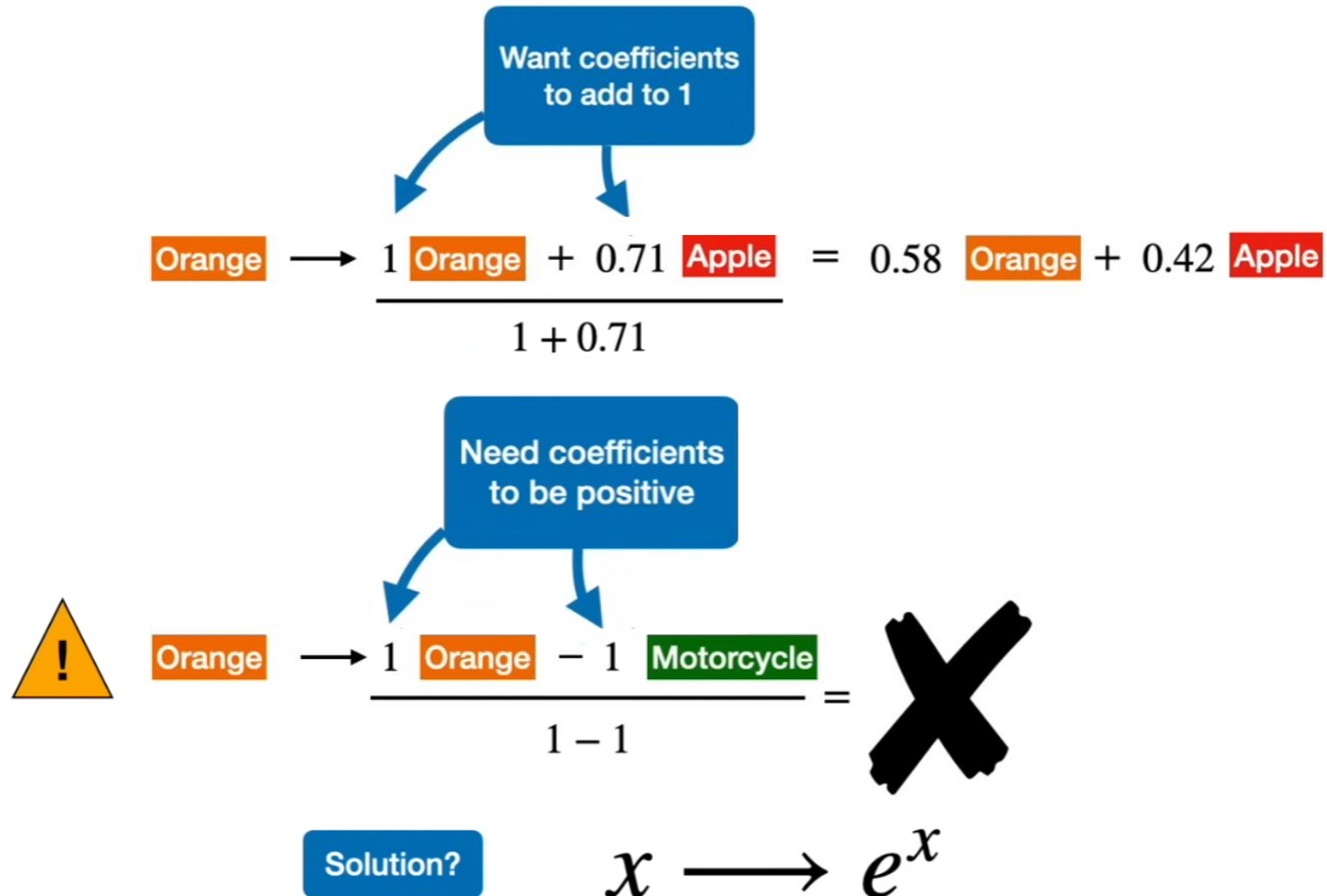
	Orange	Apple	An
Orange	1	0.71	0
Apple	0.71	1	0
An	0	0	1

$$\text{Phone} \rightarrow 1 \text{ Phone} + 0.71 \text{ Apple}$$

$$\text{Apple} \rightarrow 0.71 \text{ Phone} + 1 \text{ Apple}$$

$$\text{An} \rightarrow 1 \text{ An}$$

Normalization



Softmax

$$x \longrightarrow e^x$$

$$\text{Orange} \longrightarrow \frac{1 \text{ Orange} + 0.71 \text{ Apple}}{1 + 0.71} = 0.58 \text{ Orange} + 0.42 \text{ Apple}$$

$$\text{Orange} \longrightarrow \frac{e^1 \text{ Orange} + e^{0.71} \text{ Apple}}{e^1 + e^{0.71}} = 0.57 \text{ Orange} + 0.43 \text{ Apple}$$


$$\text{Orange} \longrightarrow \frac{1 \text{ Orange} - 1 \text{ Motorcycle}}{1 - 1} =$$


$$\text{Orange} \longrightarrow \frac{e^1 \text{ Orange} + e^{-1} \text{ Motorcycle}}{e^1 + e^{-1}} = 0.88 \text{ Orange} + 0.12 \text{ Motorcycle}$$

Softmax

	Orange	Apple	And	An
Orange	1	0.71	0	0
Apple	0.71	1	0	0
And	0	0	1	1
An	0	0	1	1

$$\text{Orange} \rightarrow 1 \text{ Orange} + 0.71 \text{ Apple}$$

$$\text{Apple} \rightarrow 0.71 \text{ Orange} + 1 \text{ Apple}$$

$$\text{And} \rightarrow 1 \text{ And} + 1 \text{ An}$$

$$\text{An} \rightarrow 1 \text{ An} + 1 \text{ And}$$

$$\text{Orange} \rightarrow 0.57 \text{ Orange} + 0.43 \text{ Apple}$$

$$\text{Apple} \rightarrow 0.43 \text{ Orange} + 0.57 \text{ Apple}$$

$$\text{And} \rightarrow 0.5 \text{ And} + 0.5 \text{ An}$$

$$\text{An} \rightarrow 0.5 \text{ An} + 0.5 \text{ And}$$

	Orange	Apple	An
Orange	1	0.71	0
Apple	0.71	1	0
An	0	0	1

$$\text{Phone} \rightarrow 1 \text{ Phone} + 0.71 \text{ Apple}$$

$$\text{Apple} \rightarrow 0.71 \text{ Phone} + 1 \text{ Apple}$$

$$\text{An} \rightarrow 1 \text{ An}$$

$$\text{Phone} \rightarrow 0.57 \text{ Phone} + 0.43 \text{ Apple}$$

$$\text{Apple} \rightarrow 0.43 \text{ Phone} + 0.57 \text{ Apple}$$

$$\text{An} \rightarrow 1 \text{ An}$$

Implementation

	Orange	Apple	And	An
Orange	1	0.71	0	0
Apple	0.71	1	0	0
And	0	0	1	1
An	0	0	1	1

$$\text{Orange} \rightarrow 1 \text{ Orange} + 0.71 \text{ Apple}$$

$$\text{Apple} \rightarrow 0.71 \text{ Orange} + 1 \text{ Apple}$$

$$\text{And} \rightarrow 1 \text{ And} + 1 \text{ An}$$

$$\text{An} \rightarrow 1 \text{ An} + 1 \text{ And}$$

$$\text{Orange} \rightarrow 0.57 \text{ Orange} + 0.43 \text{ Apple}$$

$$\text{Apple} \rightarrow 0.43 \text{ Orange} + 0.57 \text{ Apple}$$

$$\text{And} \rightarrow 0.5 \text{ And} + 0.5 \text{ An}$$

$$\text{An} \rightarrow 0.5 \text{ An} + 0.5 \text{ And}$$

an **apple** and an orange

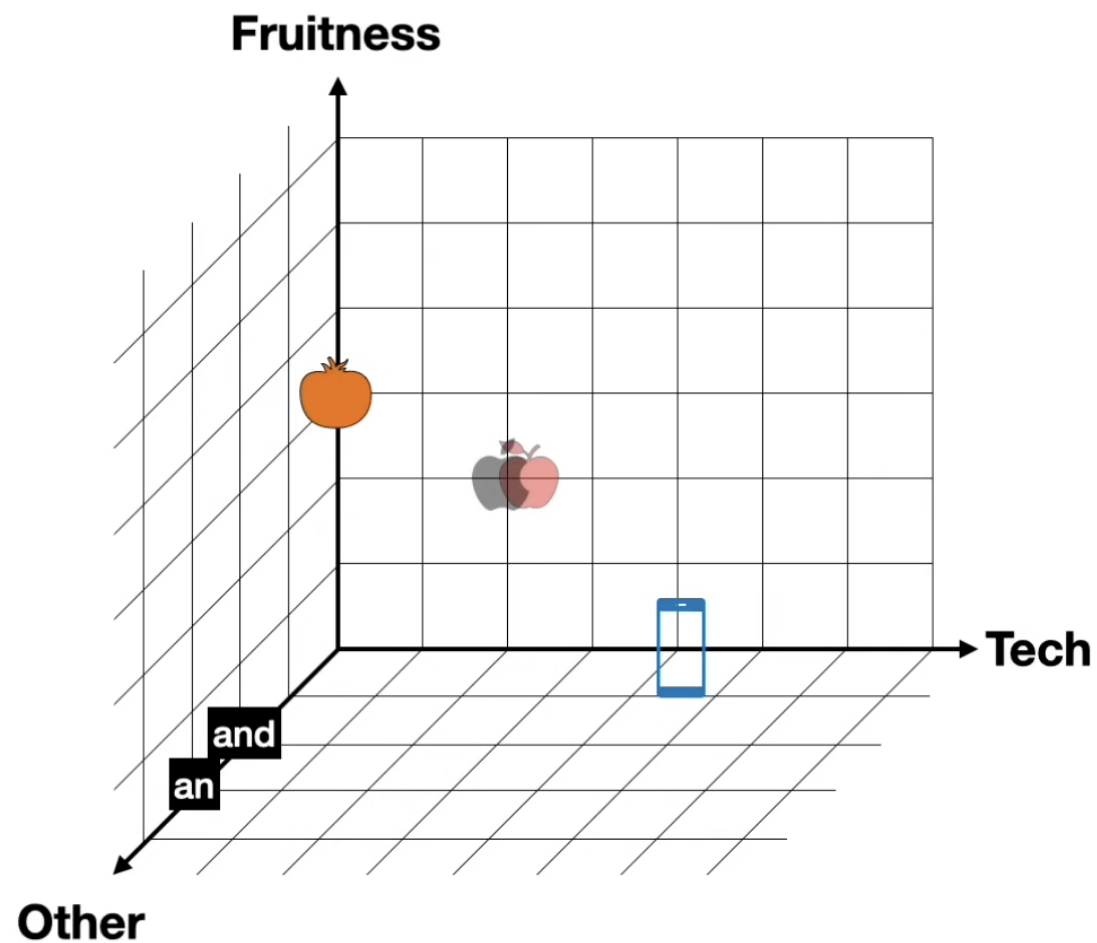
	Orange	Apple	And	An
Orange	1	0.71	0	0

$$\frac{e^1 \text{Orange} + e^{0.71} \text{Apple} + e^0 \text{And} + e^0 \text{An}}{e^1 + e^{0.71} + e^0 + e^0}$$

$$\text{Orange} \rightarrow 0.57 \text{ Orange} + 0.43 \text{ Apple}$$

$$\text{Orange} \rightarrow 0.4 \text{ Orange} + 0.3 \text{ Apple} + 0.15 \text{ And} + 0.15 \text{ An}$$

Word math



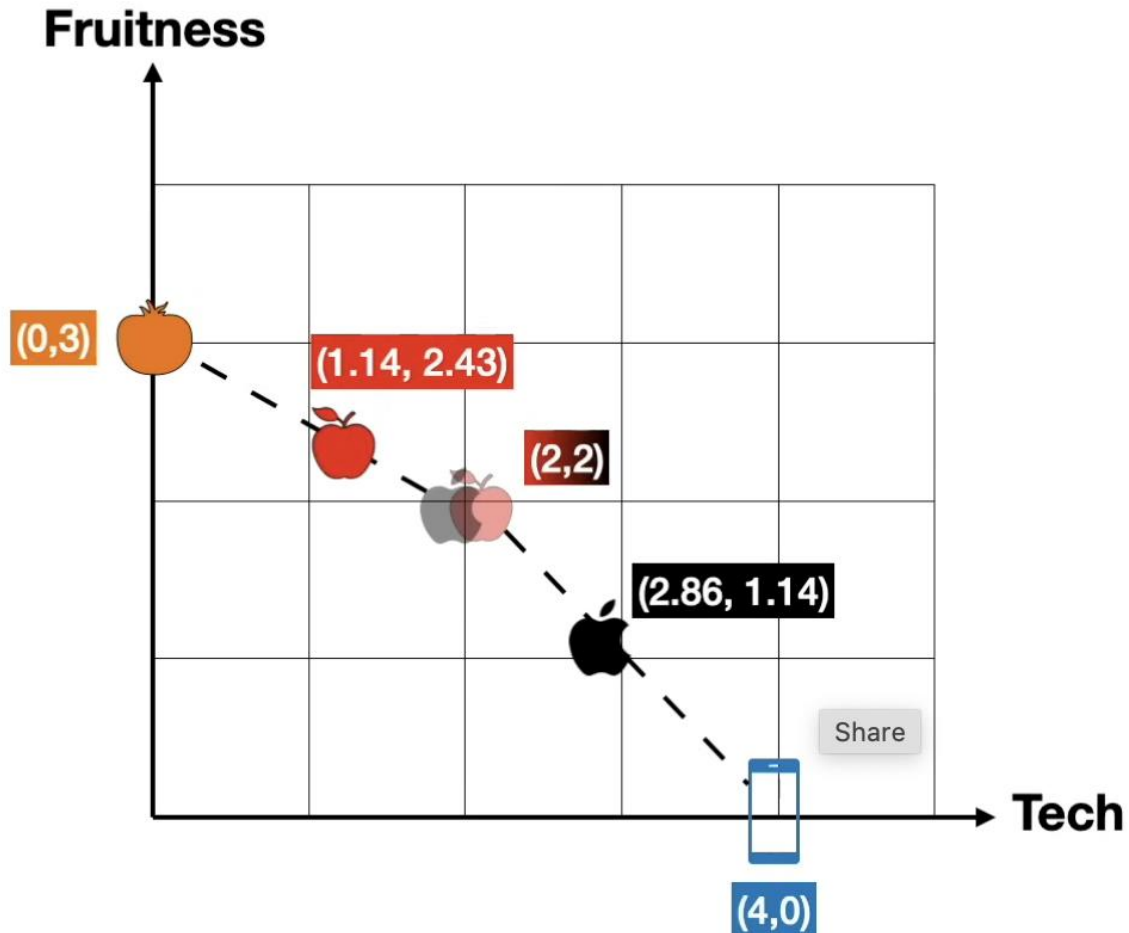
an **apple** and an orange

$$\text{Apple} \rightarrow 0.43 \text{ Orange} + 0.57 \text{ Apple}$$

an **apple** phone

$$\text{Apple} \rightarrow 0.43 \text{ Phone} + 0.57 \text{ Apple}$$

Word math



an **apple** and an orange

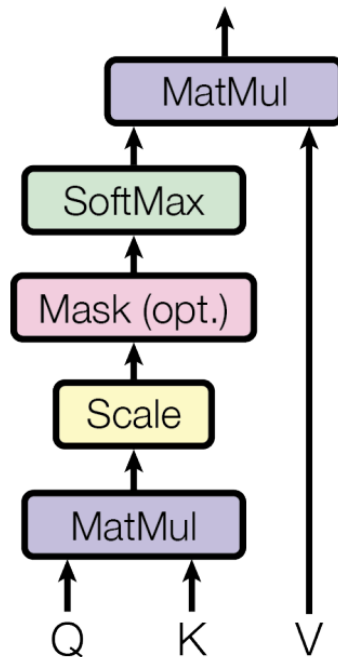
$$\text{Apple} \rightarrow 0.43 \text{ Orange} + 0.57 \text{ Apple}$$

an **apple** phone

$$\text{Apple} \rightarrow 0.43 \text{ Phone} + 0.57 \text{ Apple}$$

Query (Q), Key (K), Value (V)

Scaled Dot-Product Attention

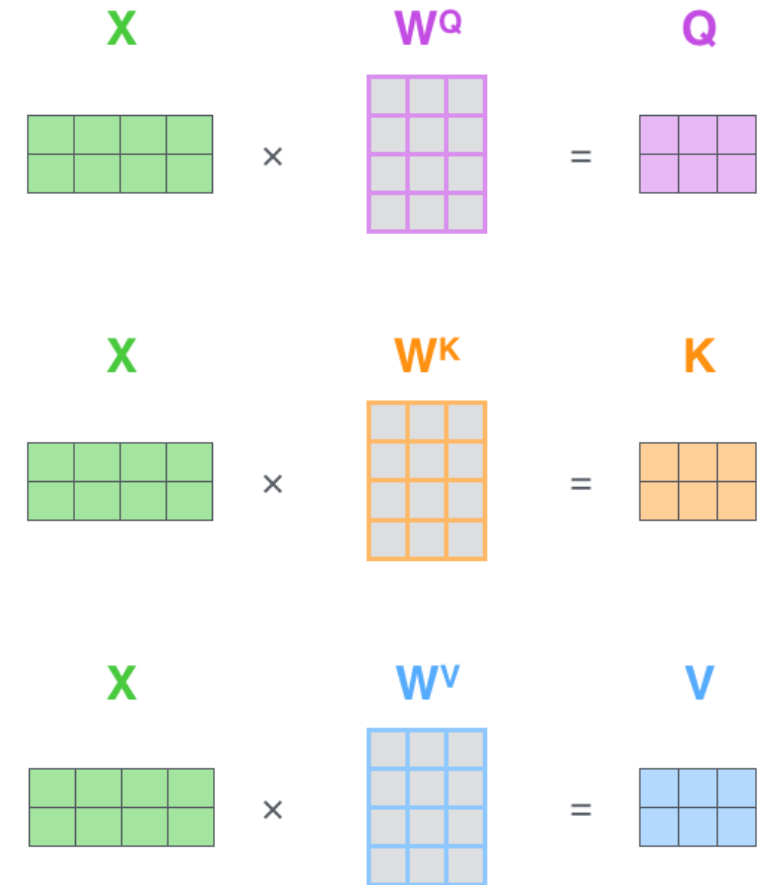


$$Q = xW_Q,$$

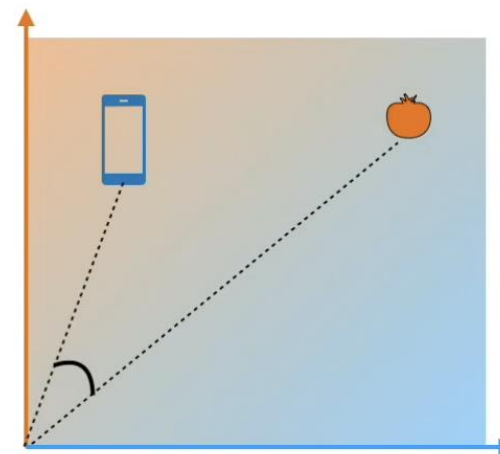
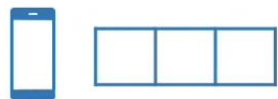
$$K = xW_K,$$

$$V = xW_V,$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

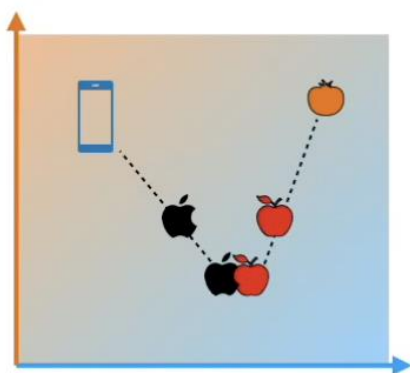


Query (Q), Key (K), Value (V)

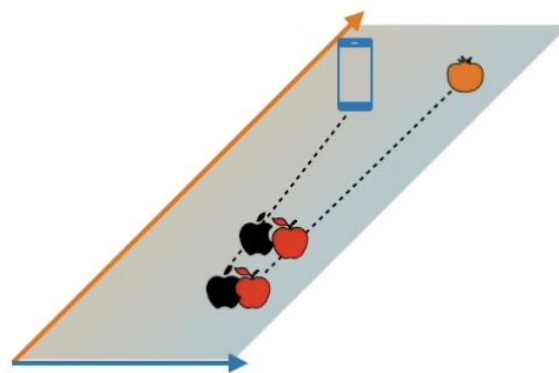


$$\text{Similarity}(\text{tomato}, \text{smartphone}) = \begin{array}{|c|c|c|} \hline & & \\ \hline \end{array} \begin{array}{|c|} \hline \\ \hline \\ \hline \end{array}$$

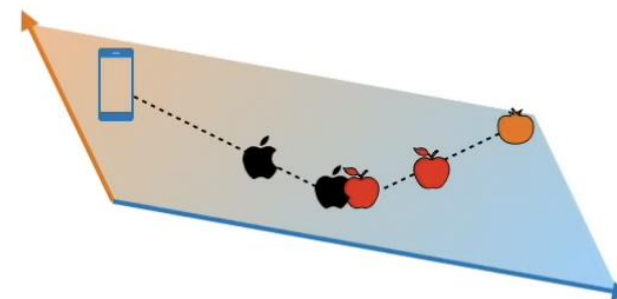
Query (Q), Key (K), Value (V)



Okay



Bad

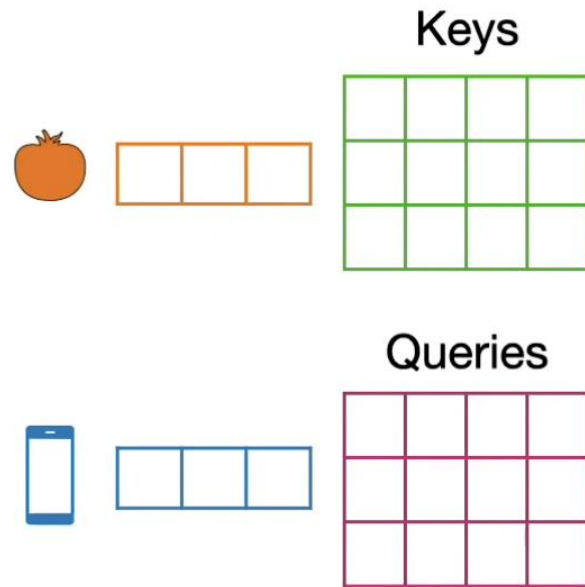


Good

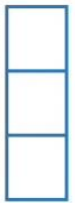
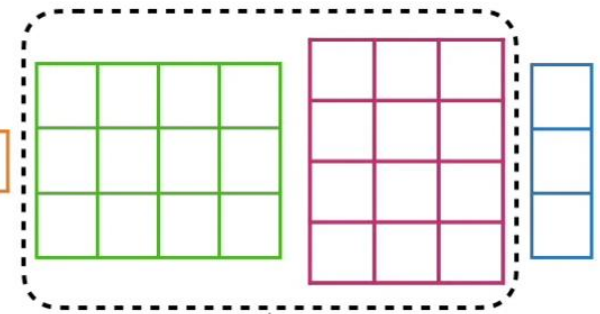
Keys

Queries

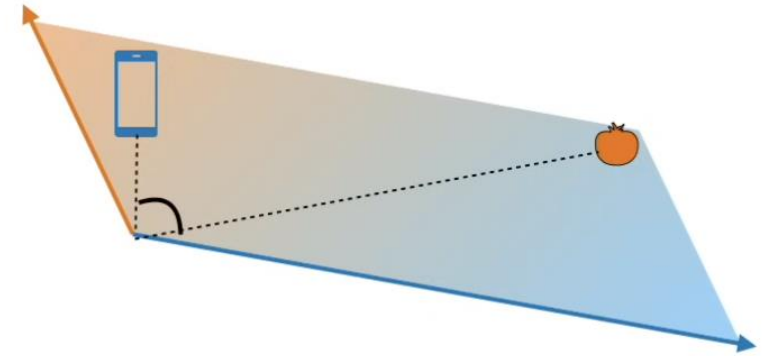
Query (Q), Key (K), Value (V)



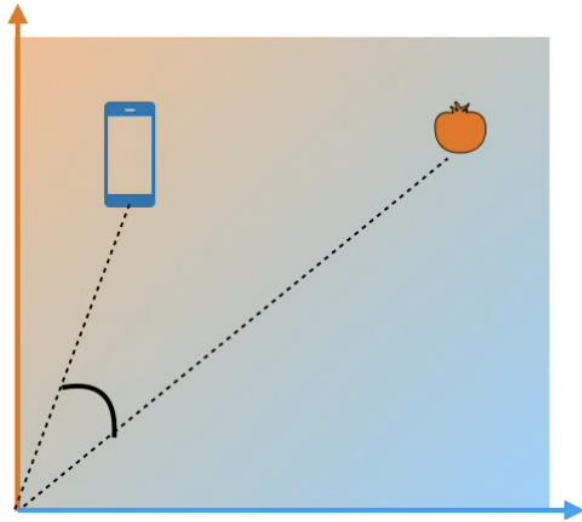
Similarity (🍅, 📱) =



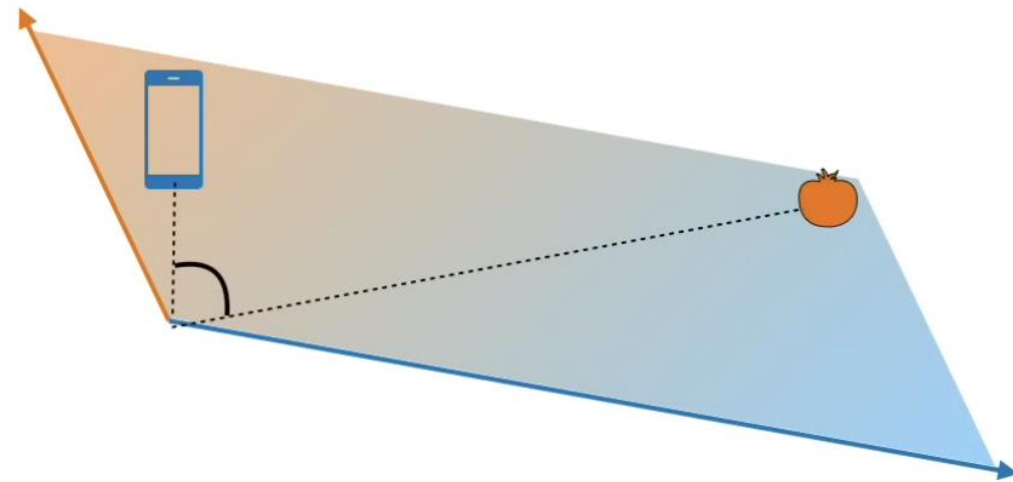
Linear transformation!



Query (Q), Key (K), Value (V)

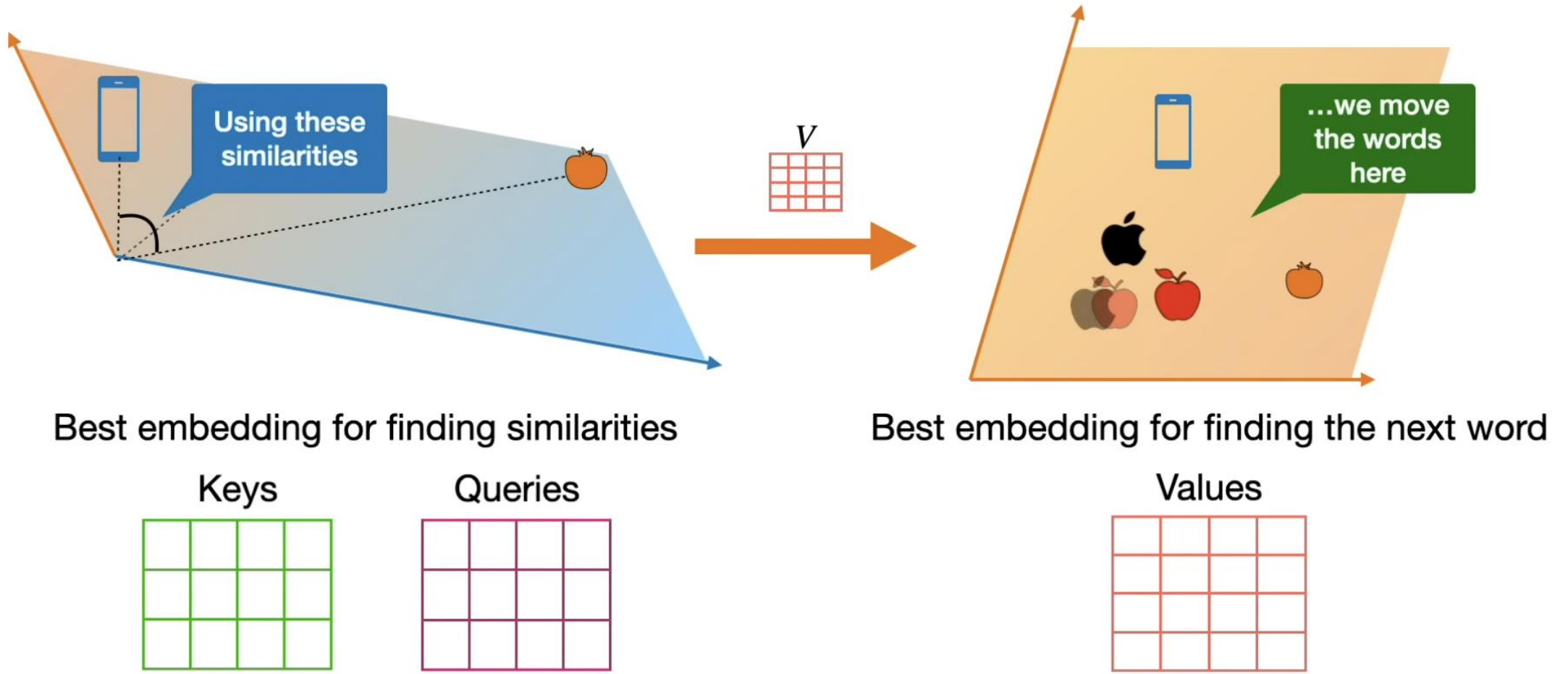


$$\text{Similarity}(\text{tomato}, \text{phone}) = \begin{array}{|c|c|c|} \hline & & \\ \hline \end{array} \begin{array}{|c|} \hline \\ \hline \end{array}$$



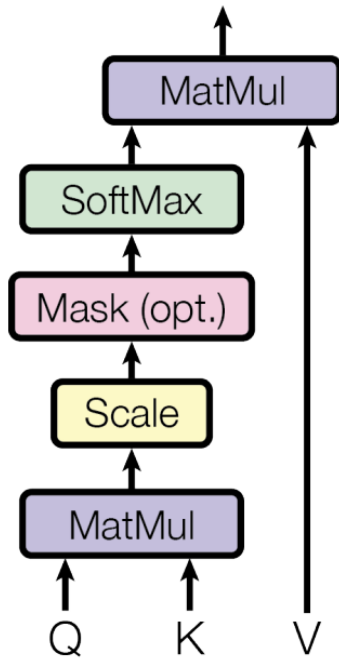
$$\text{Similarity}(\text{tomato}, \text{phone}) = \begin{array}{|c|c|c|} \hline & & \\ \hline \end{array} \begin{array}{|c|c|c|c|} \hline & & & \\ \hline \end{array} \begin{array}{|c|c|c|c|} \hline & & & \\ \hline \end{array} \begin{array}{|c|} \hline \\ \hline \end{array}$$

Query (Q), Key (K), Value (V)



Multi-head attention

Scaled Dot-Product Attention



$$Q = xW_Q,$$

$$K = xW_K,$$

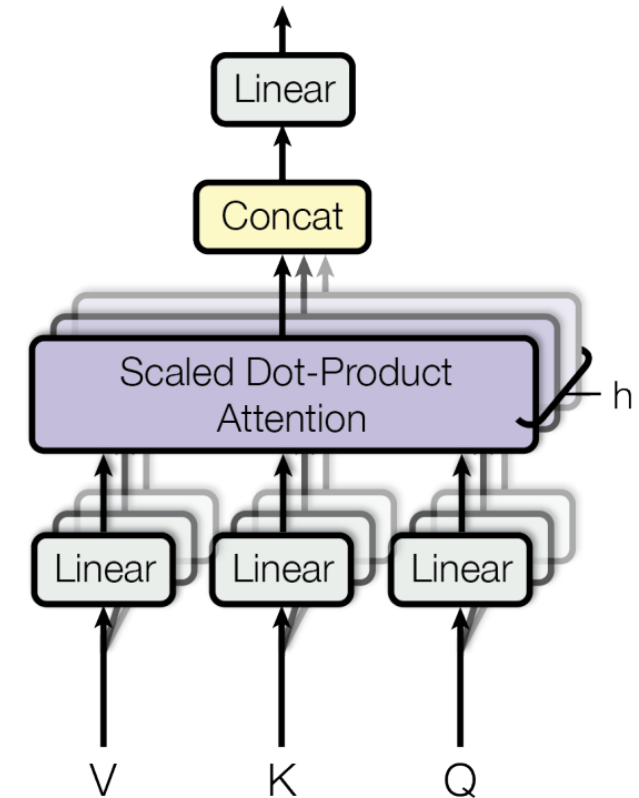
$$V = xW_V,$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

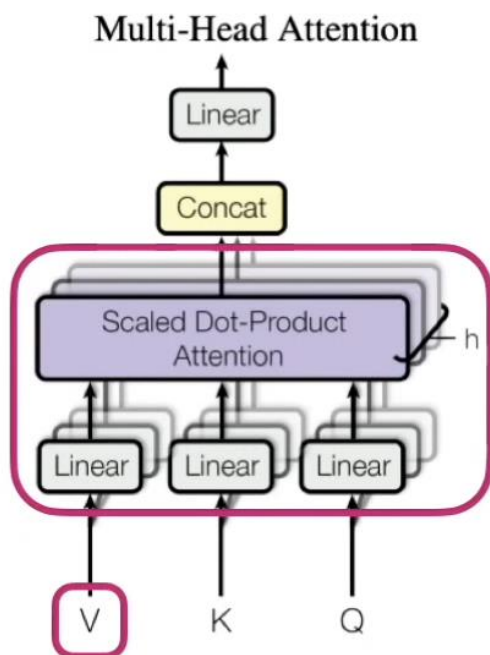
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Multi-Head Attention

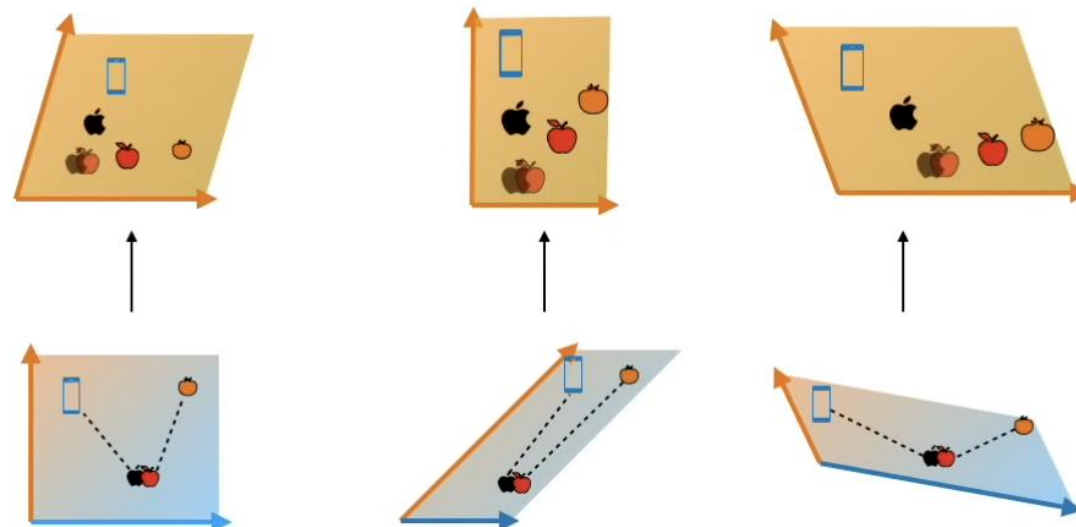
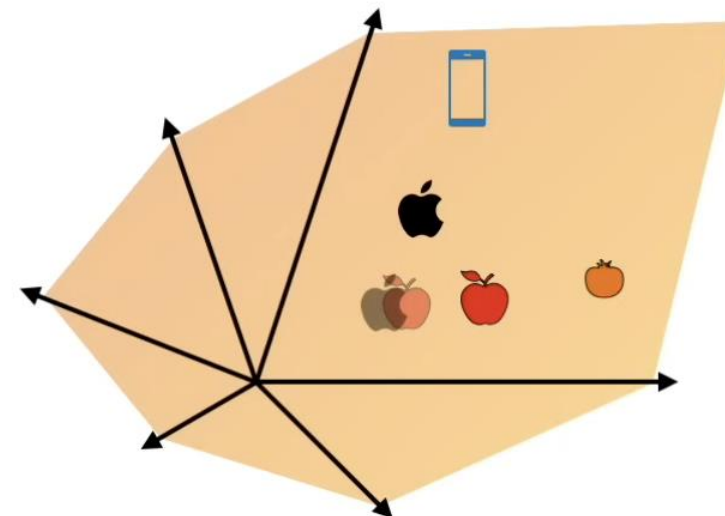
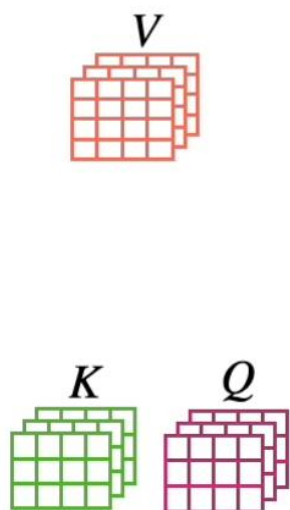


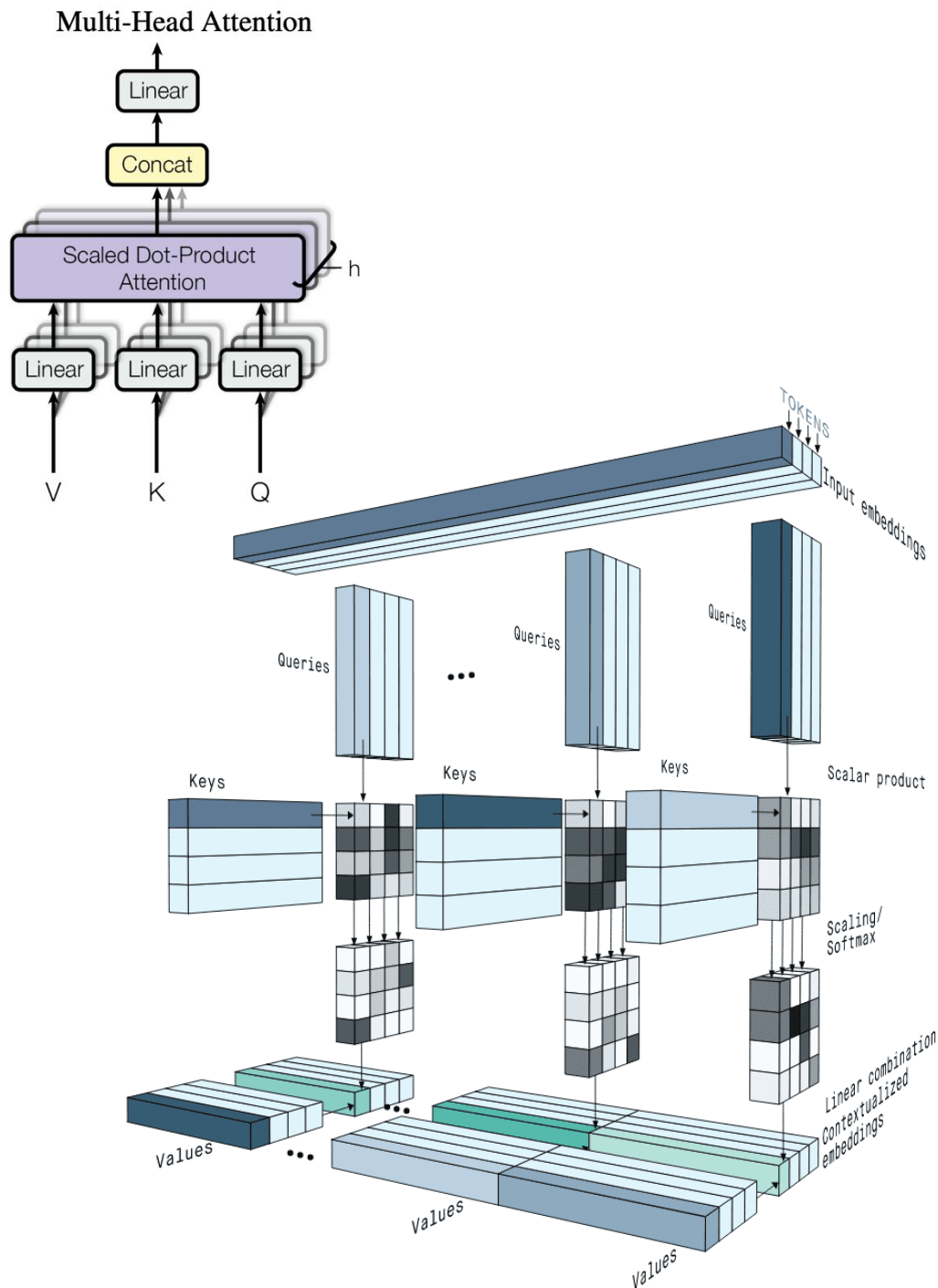
Multi-head attention



$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

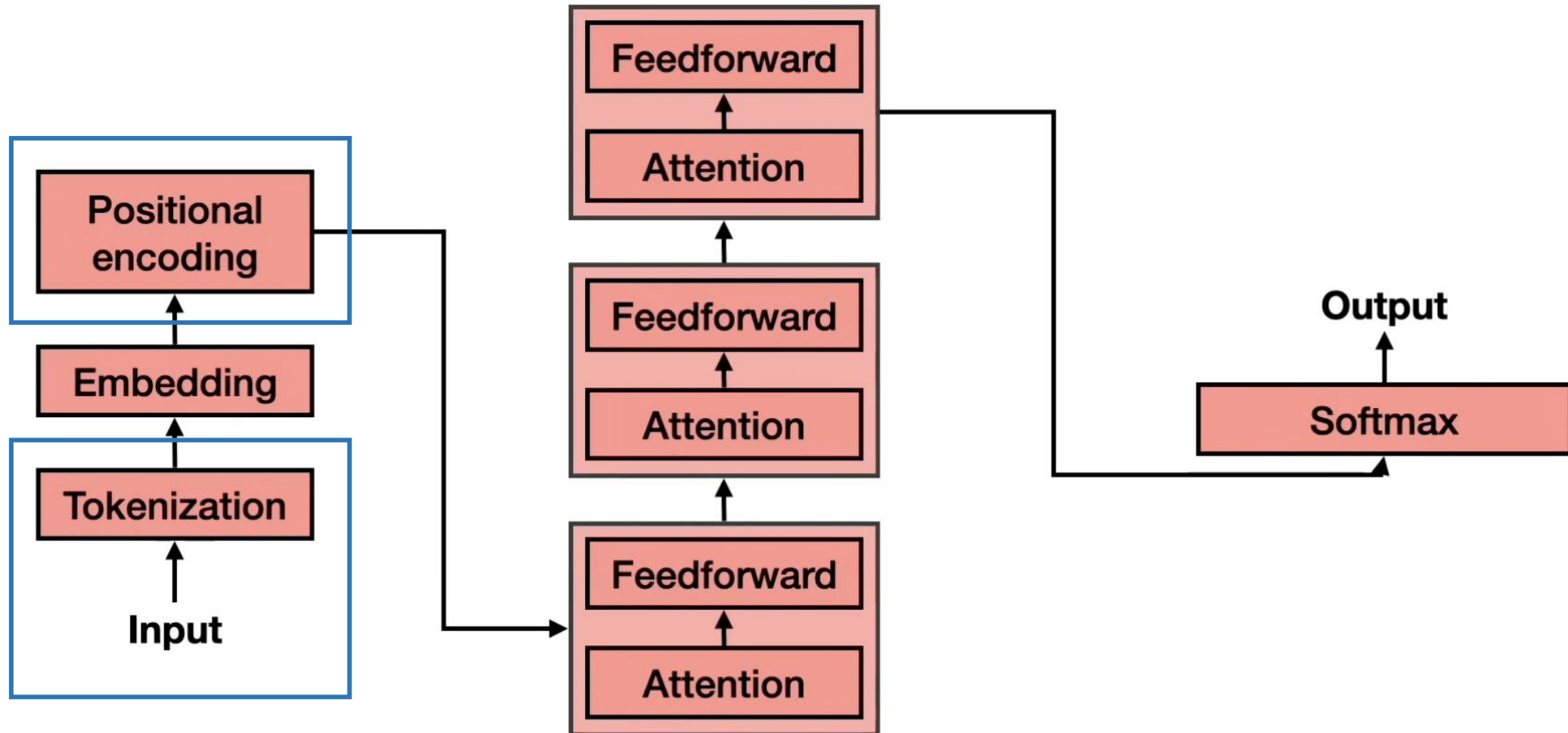




Example: multi-head attention (2 heads)

- Input: X is a matrix with shape $(4, 64)$
- Generate QKV for 2 heads:
 - For Head 1:
 - $Q_1 = X \cdot W_{Q_1}$ ($W_{Q_1} \in \mathbb{R}^{64 \times 32}$, $Q_1 \in \mathbb{R}^{4 \times 32}$)
 - $K_1 = X \cdot W_{K_1}$ ($W_{K_1} \in \mathbb{R}^{64 \times 32}$, $K_1 \in \mathbb{R}^{4 \times 32}$)
 - $V_1 = X \cdot W_{V_1}$ ($W_{V_1} \in \mathbb{R}^{64 \times 32}$, $V_1 \in \mathbb{R}^{4 \times 32}$)
- Attention calculation for each head:
 - For Head 1:
 - $\text{Score}_1 = Q_1 K_1^T \in \mathbb{R}^{4 \times 4}$
 - $\text{ScaledScore}_1 = \frac{\text{Score}_1}{\sqrt{32}} \in \mathbb{R}^{4 \times 4}$
 - $\text{AttnWeight}_1 = \text{Softmax}(\text{ScaledScore}_1) \in \mathbb{R}^{4 \times 4}$
 - $\text{Context}_1 = \text{AttnWeight}_1 \cdot V_1 \in \mathbb{R}^{4 \times 32}$
- Concatenate contexts from both heads:
 - $\text{Context}_C = \text{Concat}(\text{Context}_1, \text{Context}_2) \in \mathbb{R}^{4 \times 64}$
- Final output:
 - $\text{Output} = \text{Context}_C \cdot W_O \in \mathbb{R}^{4 \times 64}$, ($W_O \in \mathbb{R}^{64 \times 64}$)

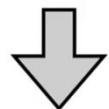
Overall Arch of GPT/Llama/DeepSeek



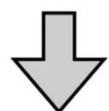
Tokenizer

LLM Tokenizer

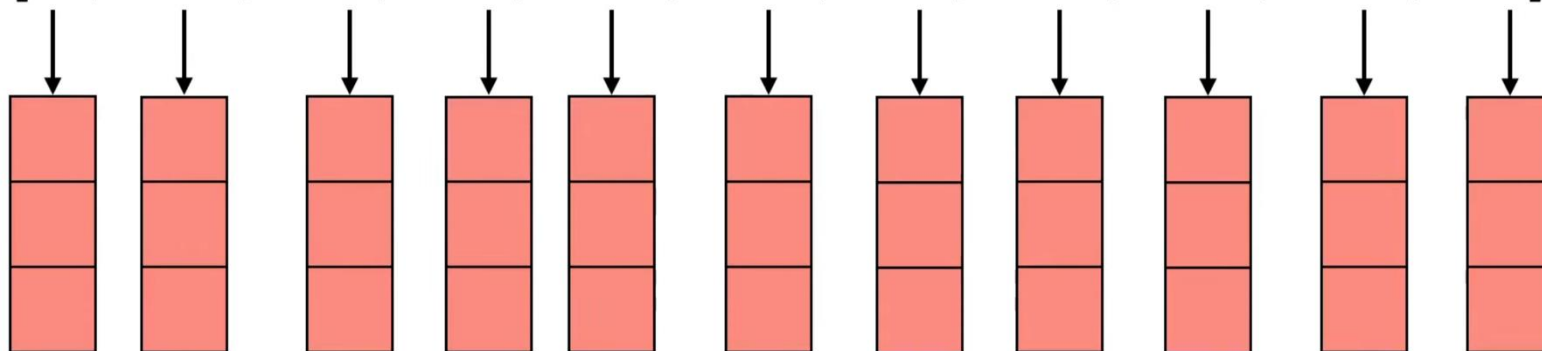
"Today is a beautiful day outside."



["To", "day", "is", "a", "beaut", "iful", "day", "out", "side", "."]



[98, 1452, 43, 15, 2932, 1709, 740, 1452, 3112, 3823, 74]



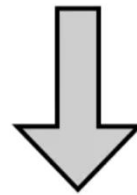
BPE Encoding

Wordpiece

Byte Pair Encoding (BPE)

Recursively merges the most frequent pairs of consecutive bytes or characters in a corpus.

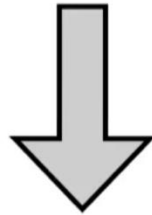
deep learning engineer



[deep, learning, engineer]

Byte Pair Encoding (BPE)

Vocabulary: [d, e, p, l, a, r, n, i, g]

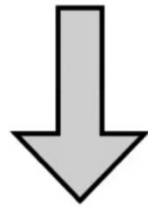


["d e e p", "l e a r n i n g", "e n g i n e e r"]

ng: 2 in: 2 ee: 2 de: 1 ep: 1 le: 1 ...

Byte Pair Encoding (BPE)

Vocabulary: [d, e, p, l, a, r, n, i, g, in]

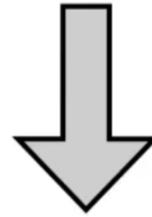


["d e e p", "l e a r n in g", "e n g in e e r"]

ee: 2	nin: 1	ing: 1	de: 1	ep: 1	...
-------	--------	--------	-------	-------	-----

Byte Pair Encoding (BPE)

Vocabulary: [d, e, p, l, a, r, n, i, g, in, ee]



["d ee p", "l e a r n i n g", "e n g i n ee r"]

dee: 1	eep: 1	le: 1	inee: 1	ea: 1	...
--------	--------	-------	---------	-------	-----

Example: GPT-2's Byte-level BPE (BBPE)

GPT-2 uses **bytes** as the base vocabulary (Byte-level BPE).

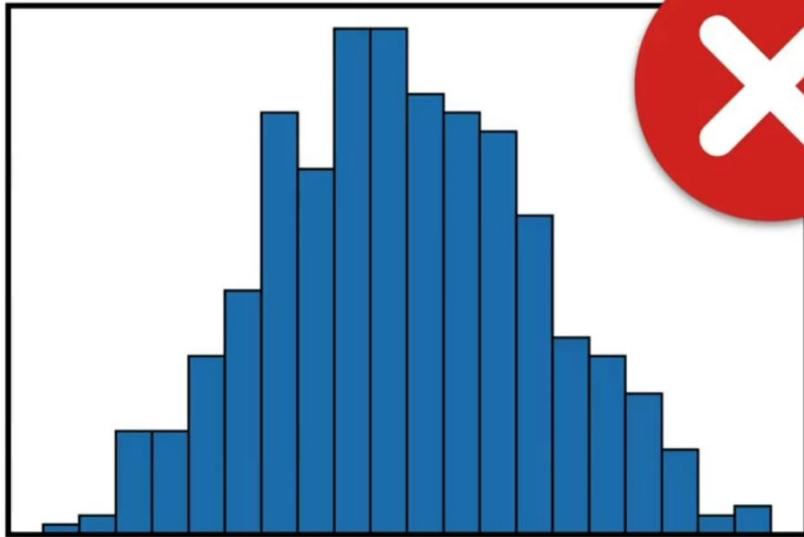
- A byte can represent 256 different values (ranging from 0 to 255 in decimal, which can cover all possible byte-level representations in the computer's encoding system).

GPT-2's vocabulary size: 50,257. Including 256 base tokens, 50,000 merge tokens, and 1 special end-of-text token (<eos>).

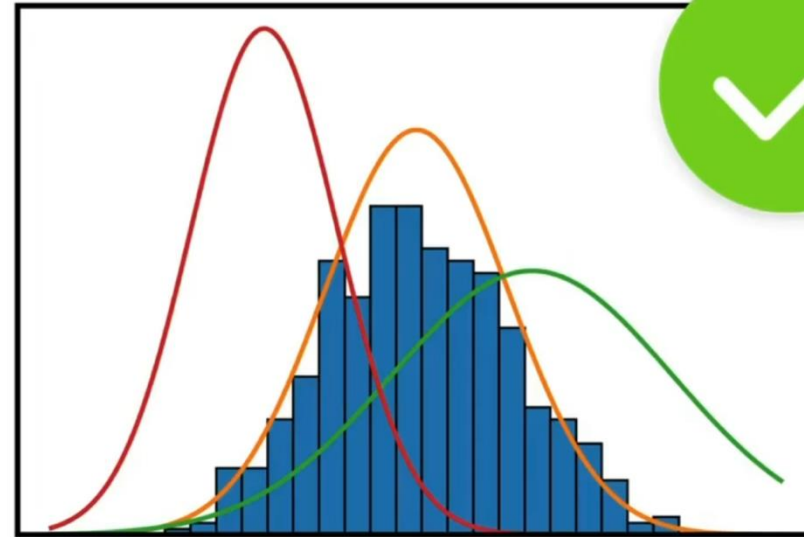
Char	Unicode	
H	00000000 01001000	English characters with 1 byte (Fill high bit with 0)
e	00000000 01100101	
l	00000000 01101100	
l	00000000 01101100	
o	00000000 01101111	
算	01111011 10010111	Chinese characters with 2 bytes
法	01101100 11010101	

WordPiece

Very similar to the BPE encoding tokenizer because it progressively learns a given number of merge rules.



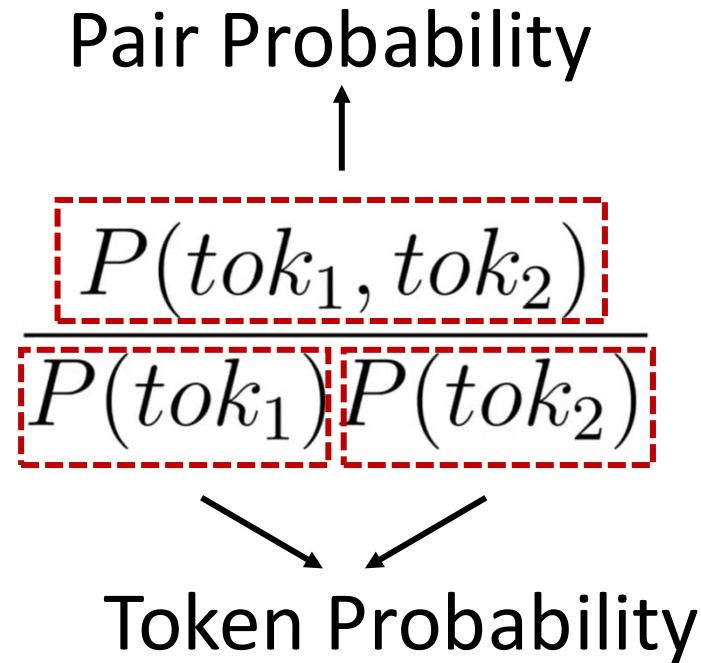
Frequency



Maximum Likelihood

WordPiece

Vocabulary: [d, e, p, l, a, r, n, i, g]
↓
["deep", "learning", "engineer"]



Most likely to co-occur and have the strongest association.
(from the perspective of likelihood).

deep learning engineer

ee: 0.117 in: 0.117 ng: 0.117 de: 0.058 ...

$$P(ee) = 2/17 = 0.1176$$

$$P(in) = 2/17 = 0.1176$$

$$P(ng) = 2/17 = 0.1176$$

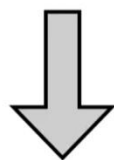
$$P(e) = 6/20 = 0.3$$

$$P(n) = 4/20 = 0.2$$

$$P(i) = 2/20 = 0.1$$

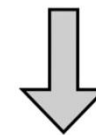
SentencePiece

deep learning engineer



[deep, learning, engineer]

Vocabulary: [d, e, p, l, a, r, n, i, g]



["d e e p", "l e a r n i n g", "e n g i n e e r"]

Japanese

トークナイザーについて楽しく学んでいただければ幸いです

Chinese

我希望您在学习
分词器过程中度
过愉快的时光

How to pre-tokenize
such language?

SentencePiece

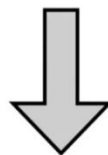
Chinese

我	希	望	你	在	学	习	分	词	器	过	程	中	度	过	...
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	-----

English

I		h	o	p	e		y	o	u		h	a	v	e	...
---	--	---	---	---	---	--	---	---	---	--	---	---	---	---	-----

deep learning engineer

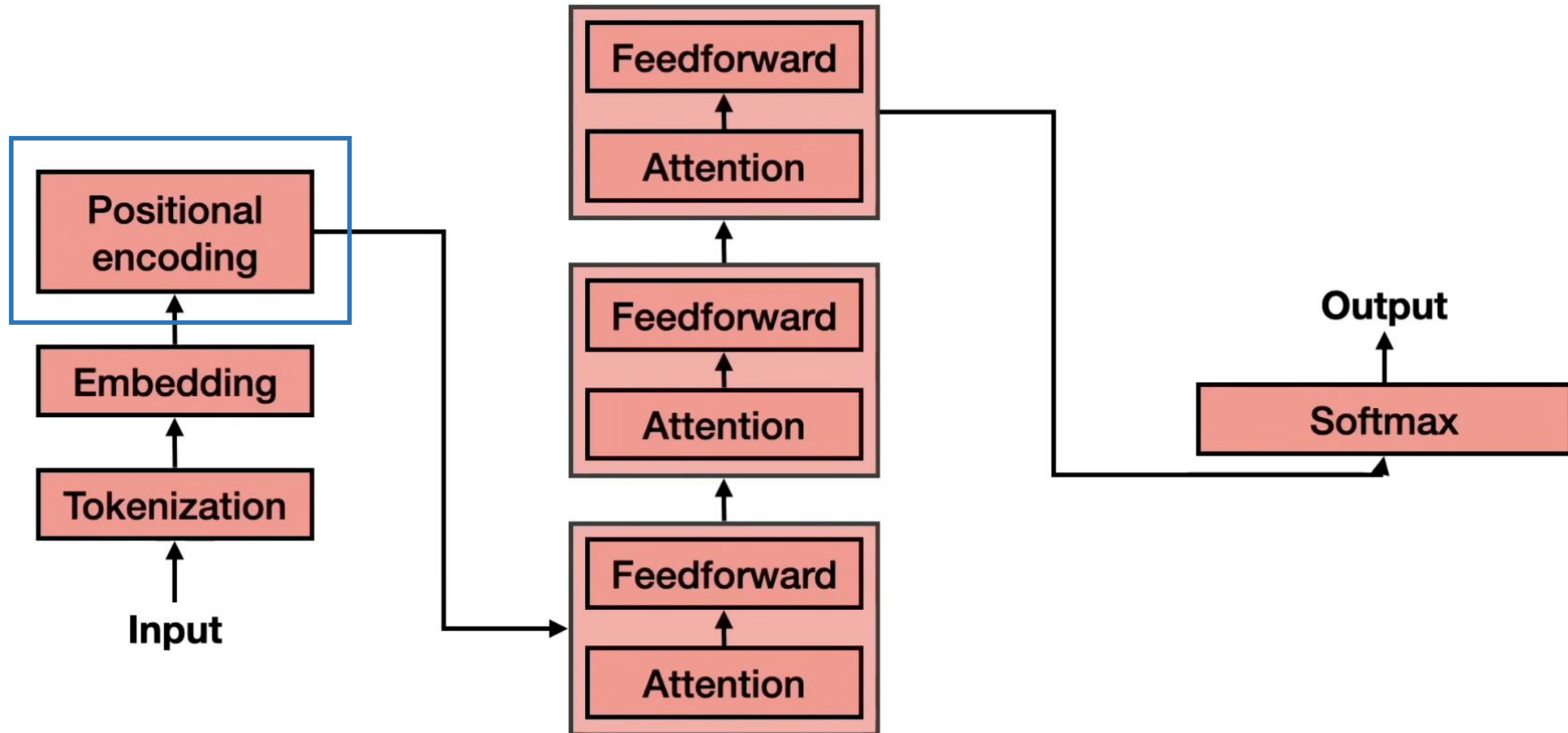


[deep, _learn, ing, _engineer]

LLM Tokenizer

Different LLMs	Scale (# Param)	Vocab Size	Tokenizer Type
LLaMA/Llama2	7B/13B/70B	32,000	BPE (SentencePiece)
Llama3	8B/70B	128,256	BPE (Byte-level)
GPT-2	S/M/L/XL	50,257	BPE (Byte-level)
Qwen2/Qwen3	1.5B/7B/14B/72B	151,936	BPE (Byte-level)
DeepSeekV3	671B (37B activated)	12,9280	BPE (Byte-level)
Kimi K3	2.8T (104B activated)	163,840	BPE (Byte-level)
Mistral	7B/123B	32,000(7B) 32,768(123B)	BPE (Byte-level)
T5	S/B/L/3B/11B	32,128	Unigram (SentencePiece)
BERT	0.1B-0.3B	30,522	WordPiece

Overall Arch of GPT/Llama/DeepSeek



Positional Encoding

Positional Encoding

Dog chased the cat.
vs.
Cat chased the dog.

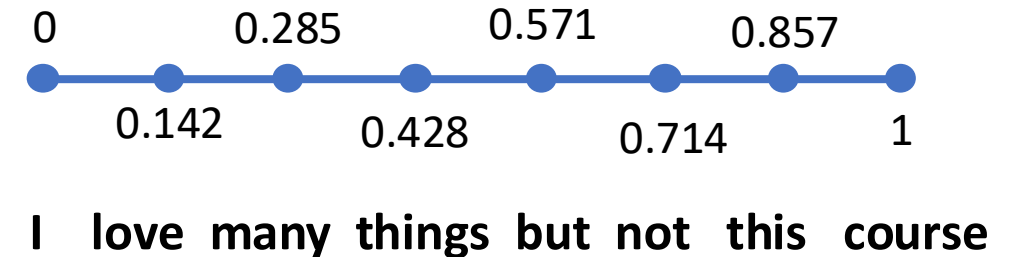
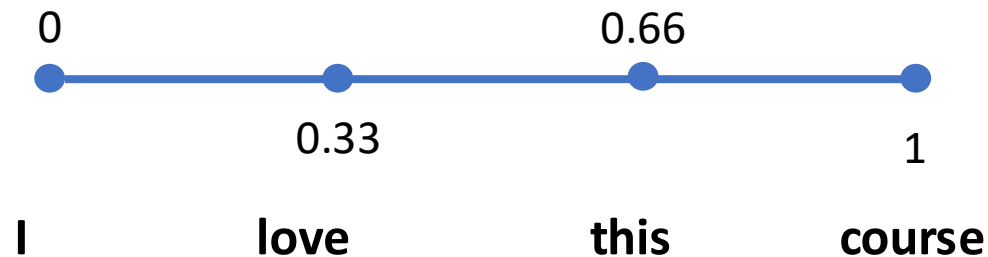
	dog	chased	the	cat
dog	1	0.71	0.5	0.9
chased	0.71	1	0.1	0.3
the	0.5	0.1	1	1
cat	0.9	0.3	1	1



Positional Encoding

Intuitive approaches:

- Linear Position Assignment (1,2,3, ...)
 - Value can be too large, unstable training
- Floats in a range ([0,1])
 - Different meaning dependent on sentence size



Positional Encoding

Properties:

- Unique positional encoding for each time step
- Reasonable notion of relative distance
- Independent of sentence size
- Deterministic



The **empress** killed the **empress**.

The dog chased the cat.

vs.

The dog, which was brown and fluffy, chased the cat.

Sinusoidal/Cosinusoidal PE

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

- Position will be a vector (instead of number)

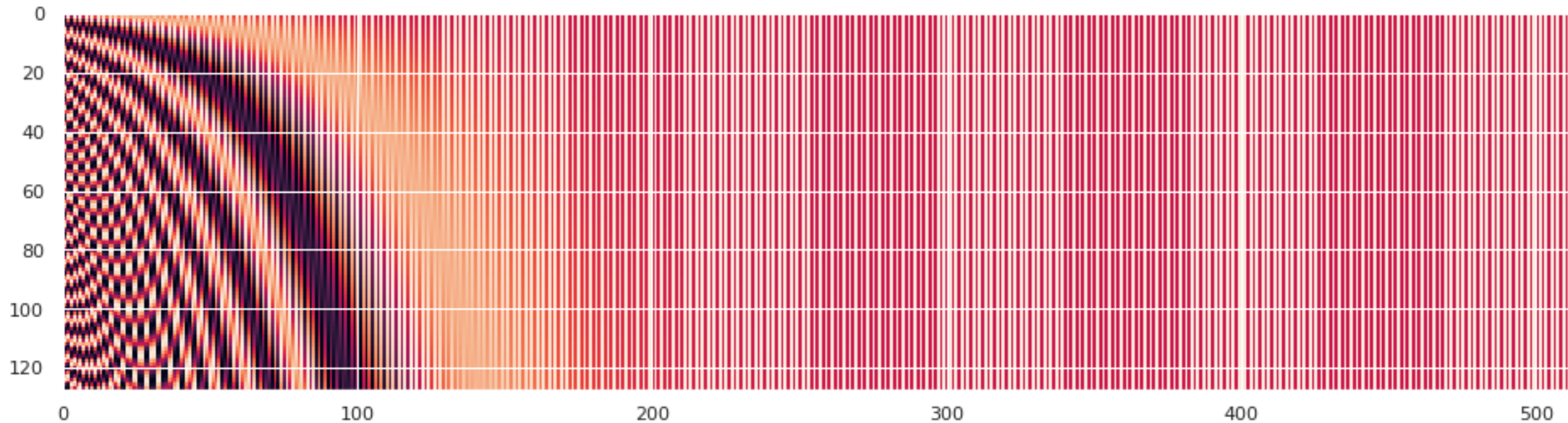
$$\vec{p}_t^{(i)} = f(t)^{(i)} := \begin{cases} \sin(\omega_k \cdot t), & \text{if } i = 2k \\ \cos(\omega_k \cdot t), & \text{if } i = 2k + 1 \end{cases}$$

$$\omega_k = \frac{1}{10000^{2k/d}}$$

$$\vec{p}_t = \begin{bmatrix} \sin(\omega_1 \cdot t) \\ \cos(\omega_1 \cdot t) \\ \\ \sin(\omega_2 \cdot t) \\ \cos(\omega_2 \cdot t) \\ \\ \vdots \\ \\ \sin(\omega_{d/2} \cdot t) \\ \cos(\omega_{d/2} \cdot t) \end{bmatrix}_{d \times 1}$$

Sinusoidal/Cosinusoidal PE

Visualization



More references

https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

<https://timodenk.com/blog/linear-relationships-in-the-transformers-positional-encoding/>

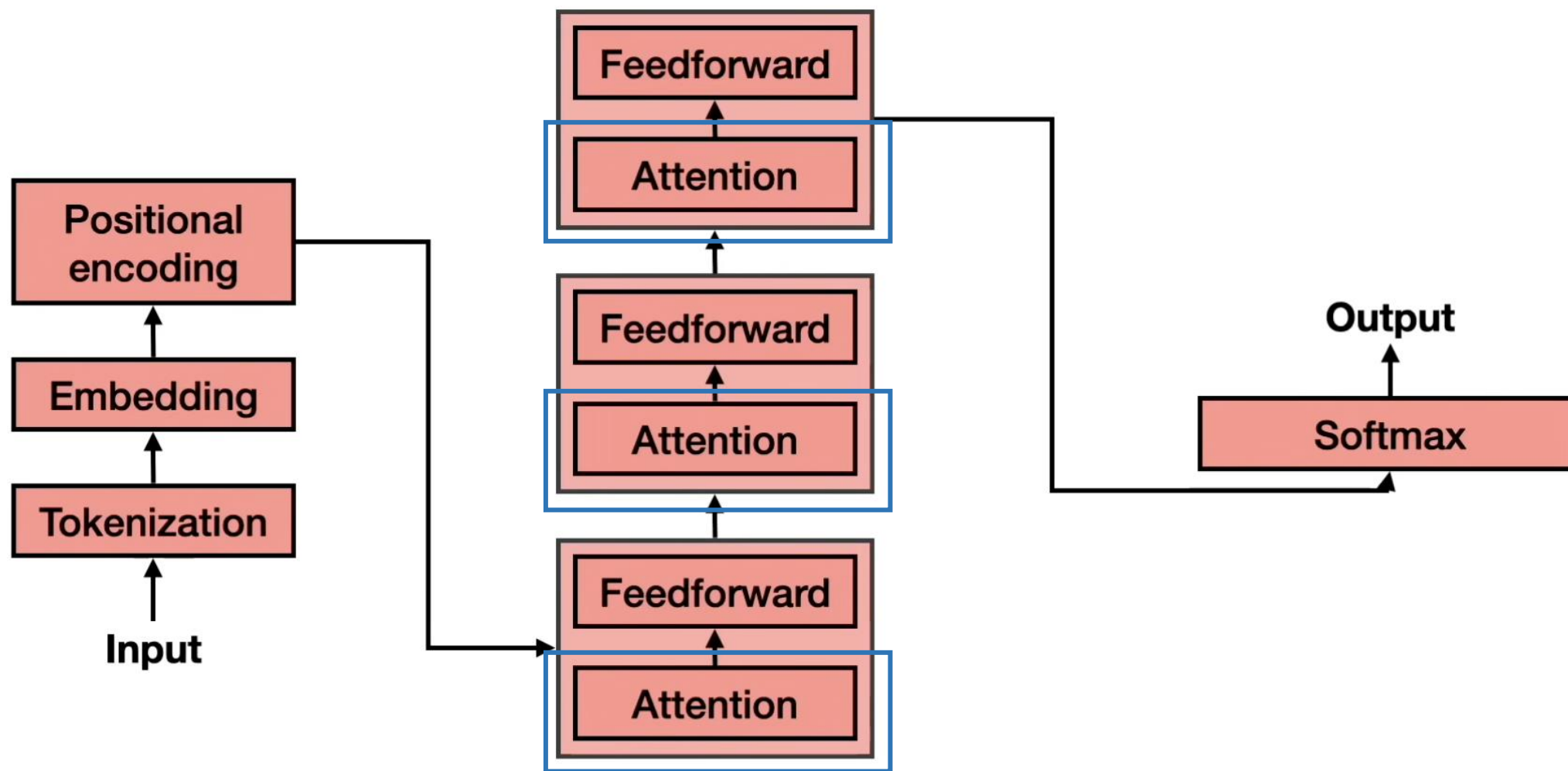
<https://jalammar.github.io/illustrated-transformer/>

<https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

<https://github.com/tensorflow/tensor2tensor/issues/1591>

https://www.reddit.com/r/MachineLearning/comments/cttefo/d_positional_encoding_in_transformer/

Overall Arch of GPT/Llama/DeepSeek



Causal Attention

Input is "Life is short, eat desert first."

Attention weight between the tokens corresponding to "Life" and "first"

	Life	is	short	eat	desert	first
Life	0.17	0.13	0.18	0.16	0.15	0.18
is	0.03	0.68	0.02	0.08	0.14	0.02
short	0.19	0.06	0.25	0.14	0.11	0.23
eat	0.15	0.21	0.14	0.16	0.17	0.14
desert	0.13	0.27	0.11	0.16	0.18	0.12
first	0.19	0.02	0.31	0.11	0.07	0.27

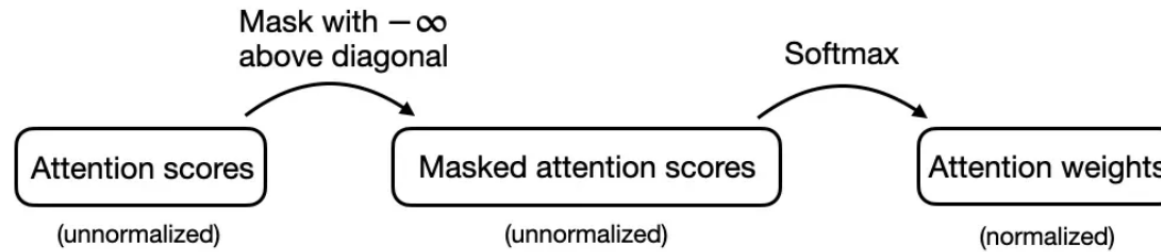
In the row for corresponding to "Life", mask out all words that come after "Life"

	Life	is	short	eat	desert	first
Life	0.17	0.13	0.18	0.16	0.15	0.18
is	0.03	0.68	0.02	0.08	0.14	0.02
short	0.19	0.06	0.25	0.14	0.11	0.23
eat	0.15	0.21	0.14	0.16	0.17	0.14
desert	0.13	0.27	0.11	0.16	0.18	0.12
first	0.19	0.02	0.31	0.11	0.07	0.27

Attention weights calculated in the previous "Self-Attention" section

- It ensures:
The outputs for a certain position in a sequence is based only on the known outputs at previous positions and not on future positions

Causal Attention



```
In [12]: mask=torch.triu(torch.ones(block_size, block_size), diagonal=1)
masked=attn_scores.masked_fill(mask.bool(), -torch.inf)
masked
```

```
Out[12]: tensor([[ -4.5107e-03,      -inf,      -inf,      -inf,      -inf,
          -inf],
         [ 1.5593e-01,  8.9439e-01,      -inf,      -inf,      -inf,
          -inf],
         [ 4.7362e-01,  2.3904e+00,  1.1067e+01,      -inf,      -inf,
          -inf],
         [-9.6320e-02, -8.0302e-01, -3.2902e+00,  4.0549e-01,      -inf,
          -inf],
         [-3.2614e-01, -1.1686e+00, -6.0539e+00,  4.3631e-01,  5.9644e+00,
          -inf],
         [ 9.0325e-02,  4.0680e-01,  1.9495e+00, -1.7108e-01, -1.8529e+00,
          3.9907e-03]], grad_fn=<MaskedFillBackward0>)
```

Causal Attention

Input is "Life is short, eat desert first."

Attention weight between the tokens corresponding to "Life" and "first"

	Life	is	short	eat	desert	first
Life	0.17	0.13	0.18	0.16	0.15	0.18
is	0.03	0.68	0.02	0.08	0.14	0.02
short	0.19	0.06	0.25	0.14	0.11	0.23
eat	0.15	0.21	0.14	0.16	0.17	0.14
desert	0.13	0.27	0.11	0.16	0.18	0.12
first	0.19	0.02	0.31	0.11	0.07	0.27

Attention weights calculated in the previous "Self-Attention" section

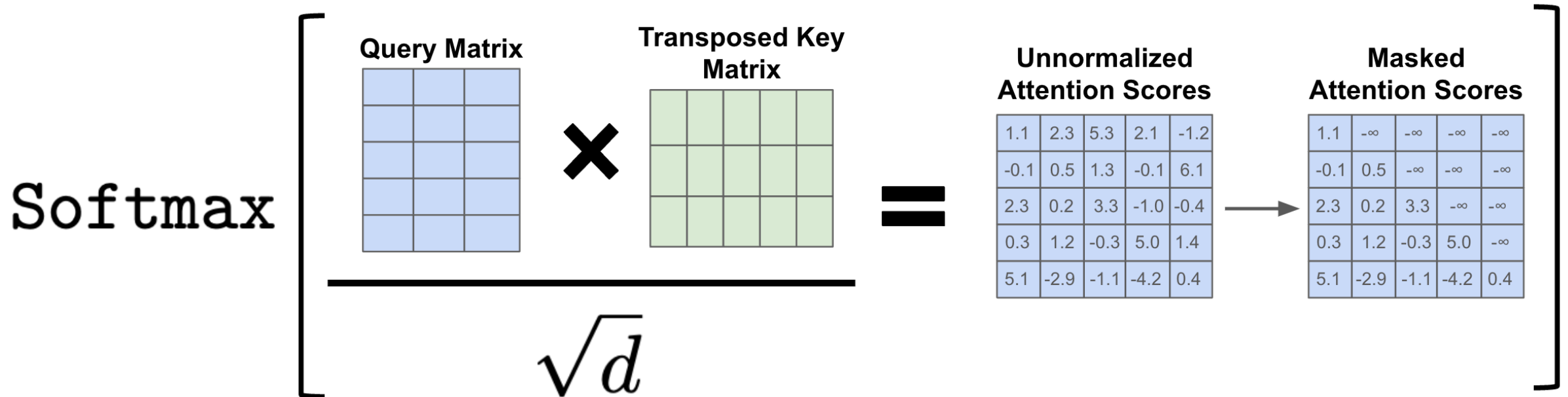
In the row for corresponding to "Life", mask out all words that come after "Life"

	Life	is	short	eat	desert	first
Life	0.17	0.13	0.18	0.16	0.15	0.18
is	0.03	0.68	0.02	0.08	0.14	0.02
short	0.19	0.06	0.25	0.14	0.11	0.23
eat	0.15	0.21	0.14	0.16	0.17	0.14
desert	0.13	0.27	0.11	0.16	0.18	0.12
first	0.19	0.02	0.31	0.11	0.07	0.27

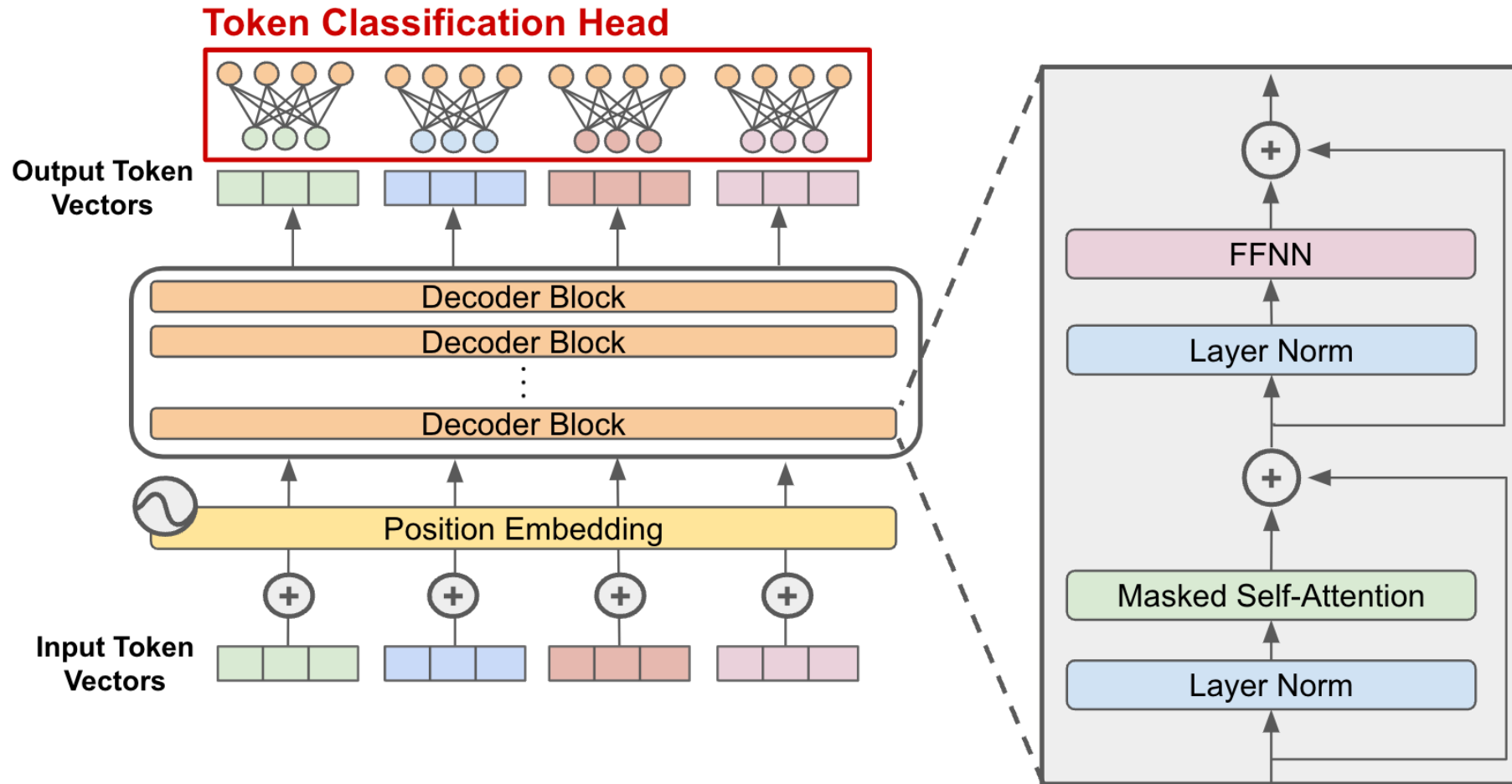
Masked out future tokens are grayed out

	Life	is	short	eat	desert	first
Life	0.17					
is	0.03	0.68				
short	0.19	0.06	0.25			
eat	0.15	0.21	0.14	0.16		
desert	0.13	0.27	0.11	0.16	0.18	
first	0.19	0.02	0.31	0.11	0.07	0.27

Causal Attention



Training phase and Generation phase



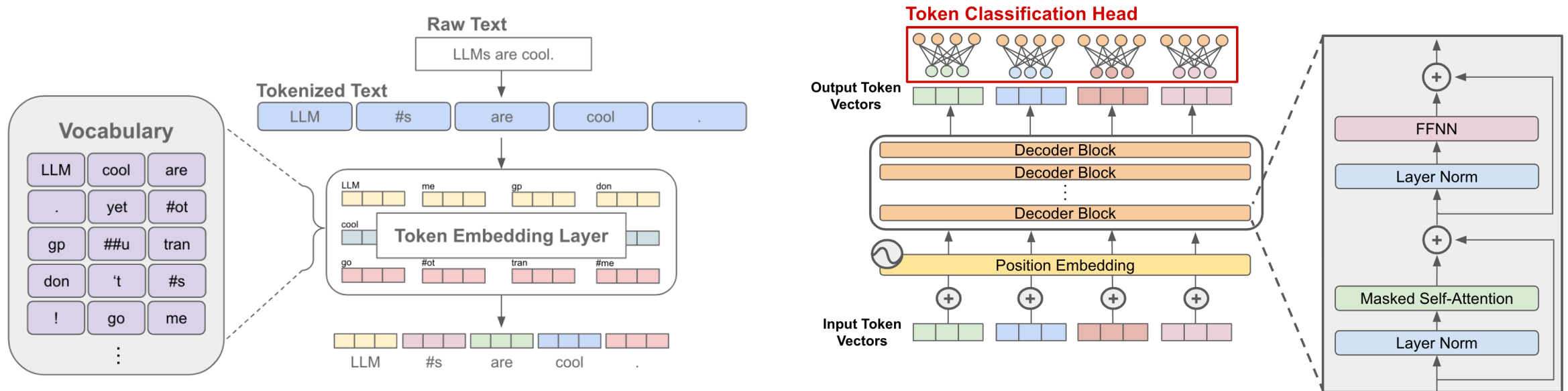
$$p(x_1, x_2, x_3, x_4) \approx p(x_1)p(x_2 \mid x_1)p_{\text{Neural}}(x_3 \mid x_1, x_2)p_{\text{Neural}}(x_4 \mid x_1, x_2, x_3)$$

Summary

Input: raw text

- Tokenizer
- Positional embedding
- Decoder block (LN, causal attention, FFN)
- Classification head

Output: next-token





Thank You!